CAPITAL UNIVERSITY · FACULTY OF ENGINEERING

Electronics and Communication Engineering Department

Digital IC Design Using FPGA

Summer Field-Training Report

Delivered online by the National Telecommunication Institute (NTI)
in collaboration with the Information Technology Industry Development Agency (ITIDA)

Student:	Zeyad Khaled Abd Elhamid	**University ID:**	2224800358
Faculty:	Engineering	**Academic year:**	Third year
Department:	Electronics & Communication Eng. (ECE)	**University:**	Capital University

Training:	Digital IC Design Using FPGA	**Mode:**	Online
Period:	6 July – 6 August 2026	**Duration:**	120 hours
Organization:	National Telecommunication Institute (NTI)	**Collaboration:**	NTI × ITIDA

Supervisor: Eng. Mohammed Salah Aboshosha

Instructors: Eng. Omar Mostafa Aladawy · Eng. Mohammed Salah Aboshosha

Abstract

ملخص التقرير

This report documents the summer field training *Digital IC Design Using FPGA*, a 120-hour online programme delivered between 6 July and 6 August 2026 by the National Telecommunication Institute (NTI) in collaboration with ITIDA, and completed as part of the third-year requirements of the Electronics and Communication Engineering Department, Faculty of Engineering, Capital University.

The training carried the author from the physics of the transistor to register-transfer design of the silicon that surrounds us. It opened with the history and economics of integrated circuits, CMOS technology, power and timing fundamentals, and a walk through the ASIC and FPGA implementation flows. The core of the programme was hands-on: ten progressive Verilog laboratories — a parameterised multiplexer, a tri-state bus driver, an 8-bit ALU, an integrated RAM–PISO–SIPO–ALU serial datapath, a bidirectional-port memory, a generic register and control unit, up/down counters in three modelling styles, an overlapping 110101 sequence detector and a function-based stream parity generator — each designed, simulated and self-checked in ModelSim/QuestaSim and preserved in a public Git repository. Supporting modules covered finite-state-machine theory and coding styles, and static timing analysis from timing arcs and unateness to setup/hold constraints, slack and XDC constraint writing. The programme closed with a team capstone: an FPGA-based automotive ECU suite of five parallel, synthesizable controllers (ABS/traction, RPM limiting, active spoiler, ultrasonic parking assist and multi-mode drive control), all verified with self-checking testbenches.

Beyond the technical content, the report reflects on the organisation, the responsibilities carried during the training, the competencies acquired, and an honest evaluation of the experience — its strengths, its difficulties and the lessons taken from both.

Keywords: Verilog, RTL design, FPGA, digital IC design, FSM, static timing analysis, ModelSim, testbench, CMOS, ECU.

Acknowledgement

I thank the National Telecommunication Institute and ITIDA for offering this training free of charge to engineering students, and my supervisor Eng. Mohammed Salah Aboshosha and instructor Eng. Omar Mostafa Aladawy for lectures that never lost the thread between transistor physics and working RTL, for patient code reviews, and for lab sheets written the way industry writes specifications. I thank my capstone teammates — Amr Waheed, Aly Hamdy, Abdelrahman Mohamed and Adham Amr — for long debugging sessions, and my family for four weeks of tolerated absence. par()

Contents

Abstract 2
Contents 2
List of Figures 4
List of Tables 5
1 Introduction 5
1.1 Overview of the training 5
1.2 Goals and objectives 6
1.3 Scope and structure of this report 6
2 The training organisation 6
2.1 The National Telecommunication Institute (NTI) 6
2.2 Collaboration with ITIDA 6
2.3 The part of the organisation I worked with 6
2.4 Delivery mode, team and responsibilities 6
3 The training programme 7
3.1 Schedule and phases 7
3.2 Workload 8
3.3 Toolchain and working method 8

4 Foundations of digital IC design 8
4.1 From vacuum tubes to systems-on-chip 8
4.2 CMOS: the technology under everything 9
4.3 Power, delay and the cost of speed 9
4.4 The ASIC design flow 10
4.5 The FPGA design flow 10
5 RTL design with Verilog 11
5.1 Abstraction levels and modelling styles 11
5.2 Data types: nets, variables, parameters 11
5.3 Procedural blocks, events and assignment semantics 11
5.4 Conditional constructs: `if`, `case`, `casex`, `casez` 12
5.5 Functions and tasks 12
5.6 Operators, literals and synthesis-aware coding 12
5.7 Sequential building blocks 13
5.8 Testbenches and verification discipline 13
6 Finite-state machines 13
6.1 What an FSM is and why it matters 13
6.2 Moore versus Mealy 13
6.3 Representation: state diagrams and ASM charts 13
6.4 Coding an FSM in Verilog 14
7 Static timing analysis 14
7.1 Why STA 14
7.2 Timing arcs and unateness 14
7.3 From transistors to setup and hold 15
7.4 Paths, constraints and slack 15
7.5 Constraint writing laboratory (XDC) 16
8 Hands-on laboratories: what I designed and verified 17
8.1 Laboratory 1 — Parameterised 2:1 multiplexer 18
8.2 Laboratory 2 — Tri-state bus driver 18
8.3 Laboratory 3 — 8-bit ALU with eight opcodes 19
8.4 Laboratory 4 — Integrated serial datapath (RAM → PISO → SIPO → ALU) 20
8.5 Laboratory 5 — Single-bidirectional-port memory 22
8.6 Laboratory 6 — Generic register and Laboratory 7 — VeriRISC control unit 23
8.7 Laboratory 8 — 2-bit up/down counter in three modelling styles 24
8.8 Laboratory 9 — Overlapping 110101 sequence detector 25
8.9 Laboratory 10 — Stream parity generator using functions 26
8.10 Laboratory 11 — Switch debouncer and edge detectors 27
9 Capstone project: FPGA-based automotive ECU suite 27
9.1 Motivation and proposal 27
9.2 Architecture 27
9.3 Subsystem design notes 28
9.4 Verification and results 30
9.5 What the project consolidated 30
10 Competencies acquired 30
11 Evaluation of the experience 31
11.1 What worked well 31
11.2 Difficulties encountered, objectively 31
11.3 Reflection on my own performance 31
12 Conclusion and future work 31
13 References 32
A Repository guide and reproduction 32
B Laboratory-to-repository mapping 32
C Daily training log 32

List of Figures

Figure 1 The five-week training plan. Lecture blocks (Introduction to Digital IC Design, the RTL Design Course, finite-state machines and static timing analysis) alternate with laboratory blocks; the capstone project proposal, implementation and review occupy week three onward, and the closing days are reserved for report writing and discussion preparation. Milestones are marked on the lower axis. 7

Figure 2 Six decades of integration. Each node marks a discontinuity: the vacuum-tube computers of the 1940s, the transistor (Bell Labs, 1947), the first integrated circuits (Kilby at Texas Instruments, 1958; Noyce at Fairchild, 1959), Moore's 1965 observation, the microprocessor era, and today's systems-on-chip and field-programmable devices. 8

Figure 3 The CMOS inverter and its steady-state behaviour. A PMOS transistor pulls the output to VDD while a NMOS transistor pulls it to GND; the two gates are driven by the same input. The table records that exactly one device conducts in steady state, which is the origin of CMOS's near-zero static power. 9

Figure 4 The ASIC flow as taught in the course, split into front-end and back-end. The front-end turns a specification into verified RTL and then into a gate-level netlist through logic synthesis; the back-end turns the netlist into geometry — floorplan, placement, clock-tree synthesis, routing — and closes timing and physical verification before tape-out. The horizontal arrow is the netlist hand-off between the two halves. 10

Figure 5 The FPGA flow used for every deliverable of the training: RTL is simulated against a testbench, synthesised into LUTs and flip-flops, placed and routed inside the fabric, and compiled to a bitstream. The dashed return path is the discipline of the course: a failing waveform sends the design back to RTL, never forward to the board. 10

Figure 6 The same specification expressed in the three Verilog modelling styles. **Structural** code instantiates gates and modules explicitly; **dataflow** code writes continuous assignments as Boolean equations; **behavioural** code describes what the circuit does inside procedural blocks and leaves synthesis to infer the structure. 11

Figure 7 The setup and hold windows. The dashed vertical line is the capturing clock edge; the data waveform must stop changing t_setup before that edge and may resume only t_hold after it. Data transitions inside either window risk metastability at the capturing flop. 15

Figure 8 A register-to-register timing path: launch flip-flop, combinational cloud, capture flip-flop, with the shared clock underneath. The start point of a path is an input port or a clock pin of a sequential cell; the end point is an output port or a data pin of a sequential cell. The four path classes taught in the module are listed below. 15

Figure 9 Block diagram of Laboratory 1 as drawn on the lab sheet: two 5-bit data inputs, a 1-bit select and a 5-bit output; the width is a module parameter, so the same source serves any address width. 18

Figure 10 Laboratory 2 block diagram. When disabled the output is not a value but an absence of one: the replicated literal `{WIDTH{1'bz}}` drives every bit of the bus to `z`. 18

Figure 11 Laboratory 3 block diagram: two 8-bit operands and a 3-bit opcode enter; an 8-bit result and the zero flag leave. The zero flag is asynchronous — pure combinational reduction equality on `in_a`. 19

Figure 12 The integrated datapath. The RAM holds 20-bit words; the PISO serialises a word under its own handshake (`en`, `ram_valid`) and even steers the RAM read address (`piso_addr`); the SIPO re-assembles the stream into `parallel_out`; the ALU decodes bit 19 as enable, bits 18:16 as opcode and the two bytes as operands, exactly as the word-format band on top shows. . . . 20

Figure 13 ModelSim extract of one serial fetch (repository VCD dump). While `en` is high the PISO loads `ram_dout` (word `0x80302`: enable, opcode ADD, operands 3 and 2) and then clocks it out on `PISO_out` one bit per cycle; `parallel_out` visibly assembles in the SIPO shift register, bit by bit, until the full word stands at the end of the burst. 21

Figure 14 ModelSim extract of consecutive executions from the same dump. `word_num` advances as each word completes; `out[7:0]` updates with each ALU execution, and `a_is_zero` pulses exactly on the words whose result is zero — the self-checking testbench's expectations reproduced on silicon-time waveforms. 21

Figure 15 Laboratory 5 block diagram. The single `data` port is an `inout` net: an input during writes, an output during reads, and high impedance otherwise — the electrical reason the port must be a net rather than a variable. 22
Figure 16 The generic register of Laboratory 6: clocked load with synchronous reset — the canonical non-blocking-assignment storage element. 23
Figure 17 How the two laboratories compose: the controller's nine control lines are pure combinational functions of `opcode`, `phase` and `zero`; the load lines (`ld_ir`, `ld_ac`) clock data into register instances, while the remaining lines steer the datapath and memory around them. 23
Figure 18 The counter interface shared by all three implementations: clock, asynchronous active-high reset, direction input and a 2-bit count. Identical ports are what make the three styles comparable under one testbench. 24
Figure 19 Moore state diagram of the overlapping detector. Each state records how much of the target pattern has been matched; a mismatch falls back to the longest state that is still a valid prefix (grey arcs), which is precisely what makes detection overlapping. State S6 asserts `seq_detected` for one clock cycle and, on a further `1`, re-enters S2 because the suffix "11" is already a prefix of the next match. 26
Figure 20 Laboratory 10 block diagram: each clock shifts `serial_in` into an 8-bit window; the parity function (a one-line XOR reduction) evaluates combinationally on the next window value and is registered onto `parity_out`. 27
Figure 21 Top-level architecture of the capstone suite. Sensors on the left feed five independent synthesizable controllers inside one clocked FPGA fabric; actuators on the right receive PWM, cut and alert signals. No controller waits for another: the five blocks evaluate every cycle, concurrently. 27

List of Tables

Table 1 Training team and points of contact 7
Table 2 Distribution of the 120 training hours 8
Table 3 Tools used during the training and their role 8
Table 4 ASIC versus FPGA — the two implementation vehicles discussed in the course 9
Table 5 The three groups of Verilog value objects, as taught on 11–12 July 11
Table 6 Blocking versus non-blocking assignment — the rule set drilled in the labs 11
Table 7 Behaviour of the case variants with respect to x and z bits 12
Table 8 Functions versus tasks 12
Table 9 Moore and Mealy machines compared, as tabulated in the FSM lecture 13
Table 10 State-encoding techniques and their trade-offs 14
Table 11 Unateness classes with the lecture's cell examples 14
Table 12 Laboratory programme at a glance 18
Table 13 Opcode encoding and operations of the laboratory ALU 19
Table 14 The five ECU modules, their interfaces and verification 28
Table 15 Competency map: what was learned and where it is evidenced 31
Table 16 Repositories accompanying this report 32
Table 17 Full mapping of sessions, laboratories, sources and concepts 32
Table 18 Day-by-day log of the training (6 July – 6 August 2026) 34

1 Introduction

1.1 Overview of the training

The summer field training *Digital IC Design Using FPGA* is an intensive, practice-first programme in digital integrated-circuit design. Over five weeks and 120 hours it follows the path a design engineer actually walks: devices and technology first, then the language of hardware description, then simulation and verification discipline, then the timing closure that decides whether a design ships, and finally a team project that puts all of it to work on a realistic problem. Lectures were delivered live online, and every

lecture block was immediately followed by laboratory work in which the same concepts were re-derived as Verilog source, testbenches and waveforms.

1.2 Goals and objectives

The training strategy pursued four goals, each of which translates into concrete outcomes evidenced later in this report:

- [**Foundations.**] Understand what an integrated circuit is, how CMOS technology makes digital logic cheap and fast, and how power, delay and area trade against each other (Chapter 4).
- [**Design language.**] Become fluent in Verilog as a *hardware* description language: data types, hierarchy, procedural blocks, blocking versus non-blocking assignment, functions, tasks, generate constructs and finite-state machines — and know which constructs synthesise into which hardware (Chapter 5, Chapter 6).
- [**Verification discipline.**] Treat simulation as the primary proof of correctness: write self-checking testbenches, read waveforms critically, and use static timing analysis to reason about setup, hold and slack without simulating (Chapter 7, Chapter 8).
- [**Engineering practice.**] Work from written specifications, keep designs in version control, deliver a team capstone project on schedule, and document everything (Chapter 8 to Chapter 9).

1.3 Scope and structure of this report

The report follows the faculty's training-report instructions: a cover page with the full identification data, a table of contents, an abstract, numbered titled sections, figures with explanatory text, and appendices. Chapter 2 introduces the hosting organisation; Chapter 3 describes the programme schedule and workload; Chapter 4 to Chapter 7 present the scientific content learned; Chapter 8 documents the hands-on laboratories with design code and simulation results; Chapter 9 presents the capstone project; Chapters Chapter 10–Chapter 12 evaluate the experience and conclude. Appendices list the code repositories, the laboratory-to-repository mapping and the daily training log.

2 The training organisation

2.1 The National Telecommunication Institute (NTI)

The National Telecommunication Institute (NTI) is the training arm of Egypt's telecommunication sector: a public institute that converts engineering graduates and students into industry-ready specialists in communications, networks, embedded systems and chip design. NTI runs its programmes as condensed versions of real engineering work — specification sheets, design reviews, tool flows and deadlines — which is exactly the culture encountered during this training: lab sheets read like industrial design specifications, and deliverables were reviewed against them.

2.2 Collaboration with ITIDA

The programme was delivered in collaboration with the Information Technology Industry Development Agency (ITIDA), the agency responsible for developing Egypt's electronics and embedded-software industry. The NTI × ITIDA frame gives the training its explicit purpose: to build a national pool of engineers able to work in digital IC design — a field the initiative treats as strategic. For students this meant curricula written against current industry flows (RTL → synthesis → timing closure) rather than against exam syllabi.

2.3 The part of the organisation I worked with

Within NTI, the training belongs to the embedded systems and VLSI training track. The track supplied the lecture material (*Introduction to Digital IC Design* and the *RTL Design Course*, plus FSM and static-timing-analysis modules), the laboratory sheets (Labs 1–9 and the counter lab), the simulation environment guidance (ModelSim/QuestaSim project flows, with Icarus Verilog and Vivado as alternatives), and the project-recognition scheme under which the capstone of Chapter 9 was submitted and reviewed.

2.4 Delivery mode, team and responsibilities

The training ran fully online: live lectures with shared slides and whiteboard, followed by self-paced laboratory work in a local ModelSim/QuestaSim installation, with design sources kept in Git repositories and reviewed asynchronously by the instruction team.

Role	Name	Contribution
Supervisor	Eng. Mohammed Salah Aboshosha	Overall supervision; Verilog and FSM lectures; lab reviews; STA module (co-author)
Instructor	Eng. Omar Mostafa Aladawy	*Introduction to Digital IC Design* course: devices, CMOS, power, ASIC/FPGA flows
Instructor	Eng. Mohammed Salah Aboshosha	*RTL Design Course*, procedural blocks, functions/tasks, counter and FSM labs
STA module	Eng. Ahmed Abdelazeem (co-author)	Static timing analysis material: arcs, unateness, constraints, slack
Organisation	NTI × ITIDA	Curriculum, lab sheets, project recognition, discussion sessions

Table 1: Training team and points of contact

My responsibilities as a trainee were, in the order the programme demanded them: to attend and prepare for lectures; to implement every laboratory from its specification sheet without reference solutions; to write and extend testbenches until each design passed its self-checks; to maintain a personal repository of all designs, testbenches and simulation projects; to co-design two of the five capstone ECU modules and verify them; and finally to write this report and prepare for the faculty discussion.

3 The training programme

3.1 Schedule and phases

The programme divides naturally into four phases, as the timeline of Figure Figure 1 shows. The first week builds foundations: the history and economics of integrated circuits, CMOS device physics, power and timing fundamentals, and the ASIC and FPGA implementation flows, together with the opening chapters of the RTL course (logic gates, number systems, module syntax and modelling styles). The second week is the laboratory core: data types and hierarchy, procedural blocks and assignment semantics, then Labs 3–8 — multiplexer, tri-state driver, ALU, controller, register and memory — followed by the case-statement variants, functions and tasks, and the FSM lecture with its state-diagram and encoding techniques. The third week deepens both threads: counters in three modelling styles, Mealy/Moore edge detectors, the 110101 sequence detector, a switch-debouncer design, and the static-timing-analysis module closed by a constraint-writing laboratory in XDC; the same week carries the capstone proposal (19 July) and submission (23 July). The remaining days until 6 August were used to polish the project presentation, re-verify all waveforms, and prepare this report and the faculty discussion.

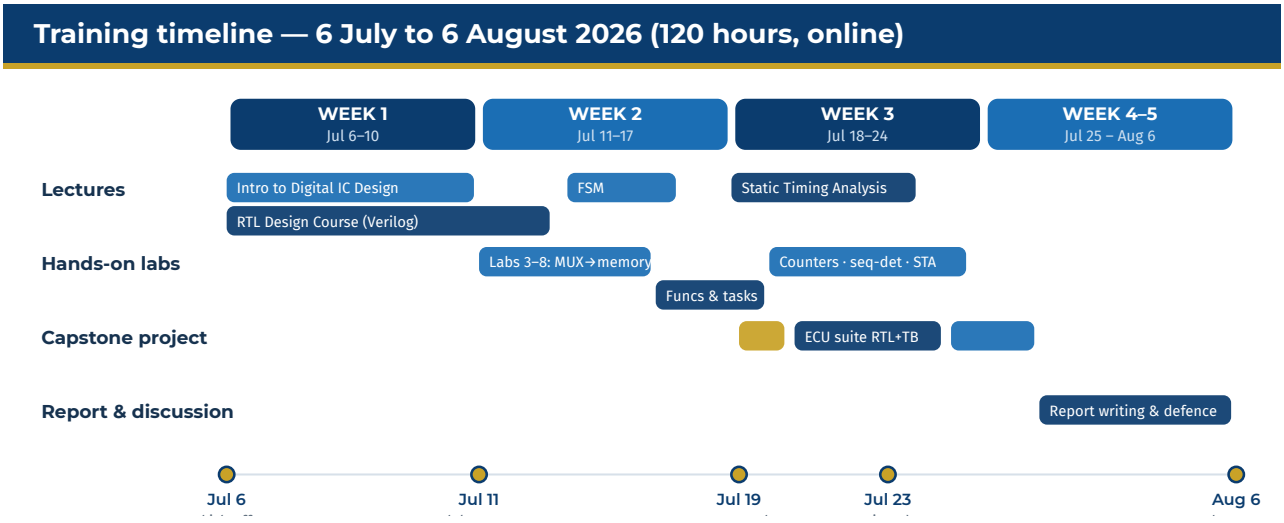


Figure 1: The five-week training plan. Lecture blocks (Introduction to Digital IC Design, the RTL Design Course, finite-state machines and static timing analysis) alternate with laboratory blocks; the capstone project proposal, implementation and review occupy week three onward, and the closing days are reserved for report writing and discussion preparation. Milestones are marked on the lower axis.

Activity	Hours	Evidence in this report
Live lectures and reviewed material	48	Chapter 4 to Chapter 7
Laboratory implementation and simulation	48	Chapter 8, Appendix B
Capstone project (team of five)	16	Chapter 9
Report writing, review, discussion prep	8	This document; Appendix C
Total	**120**	—

Table 2: Distribution of the 120 training hours

Tool	Role in the training
ModelSim / QuestaSim 10.1d	Primary simulator: prepared `.mpf` projects per lab, waveform review, VCD dumps
Icarus Verilog + GTKWave	Command-line cross-checks of every lab (`iverilog -o sim.vvp src/*.v tb/*.v`)
AMD Vivado	Synthesis experiments and the XDC constraint laboratory of Section 7.5
Git / GitHub	Version control of all designs; two public repositories (Appendix A)
Verilog-2001 / SystemVerilog	Description language of all labs and the capstone

Table 3: Tools used during the training and their role

3.2 Workload

The 120 contracted hours split roughly equally between taught material, supervised laboratory work and project work:

A day-by-day log of sessions, topics and hours is given in Appendix C it is consistent with the commit history of the laboratory repository, whose directories are named by session date (11_7, 12_7, … 22_7).

3.3 Toolchain and working method

All design work was performed with a professional flow mirrored at home:

The working method enforced by the lab sheets — and adopted for the whole training — was: read the specification and its block diagram; name ports exactly as the diagram does; write the RTL; simulate against the supplied testbench; extend the testbench with self-checks; only then commit. Waveform screenshots and console logs were kept as evidence for each lab, and several of them appear in Chapter 8.

4 Foundations of digital IC design

This chapter summarises the *Introduction to Digital IC Design* course (Eng. Omar Mostafa Aladawy): why chips exist, how CMOS makes them possible, what limits them (power, delay), and how a design travels from specification to silicon or to an FPGA bitstream.

4.1 From vacuum tubes to systems-on-chip

Figure Figure 2 places the course's historical introduction on a single axis. The engineering content of that history is Moore's law: Gordon Moore observed in 1965 that the number of transistors per chip doubles roughly every two years, and the industry has since treated that observation as a roadmap. The

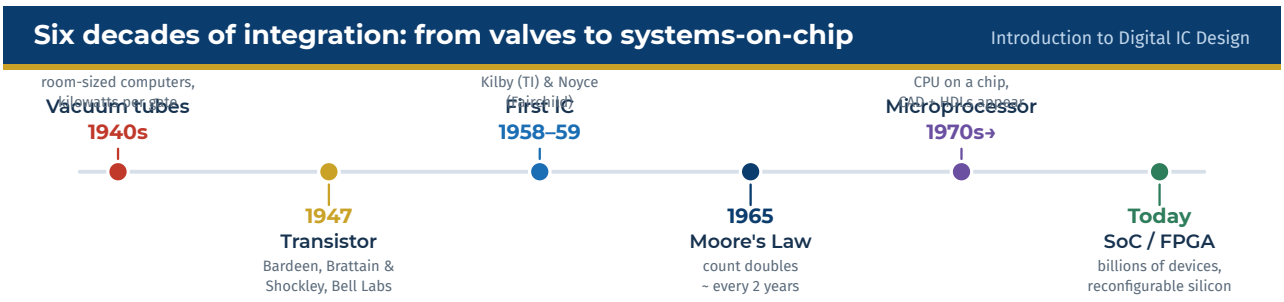
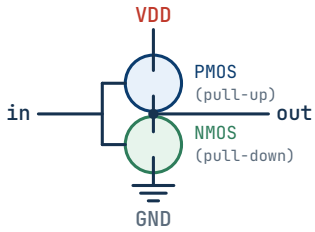


Figure 2: Six decades of integration. Each node marks a discontinuity: the vacuum-tube computers of the 1940s, the transistor (Bell Labs, 1947), the first integrated circuits (Kilby at Texas Instruments, 1958; Noyce at Fairchild, 1959), Moore's 1965 observation, the microprocessor era, and today's systems-on-chip and field-programmable devices.

Aspect	ASIC	FPGA
Meaning	Application-specific IC	Field-programmable gate array
Fabric	Fixed at mask time	Configurable logic blocks, programmable routing
Unit cost / NRE	Low unit cost, very high NRE	No NRE, higher unit cost
Performance, power	Best	Good; overhead of programmability
Time to market	Months (tape-out)	Hours (bitstream)
Role in training	Reference flow (Section 4.4)	Working vehicle for all labs and the capstone

Table 4: ASIC versus FPGA — the two implementation vehicles discussed in the course

The CMOS inverter: complementary pull-up / pull-down pair CMOS technology



Steady-state behaviour

in	out	ON device
0	1	PMOS
1	0	NMOS

Exactly one transistor conducts in steady state → near-zero static power; power is spent only on switching.

Figure 3: The CMOS inverter and its steady-state behaviour. A PMOS transistor pulls the output to VDD while a NMOS transistor pulls it to GND; the two gates are driven by the same input. The table records that exactly one device conducts in steady state, which is the origin of CMOS's near-zero static power.

consequences a designer feels are economic as much as physical — transistors are effectively free, wires and power are not — which is why modern courses start with technology and power before any logic is written. The course also classified ICs (analog, digital, mixed-signal; SSI through VLSI) and contrasted the two implementation vehicles used throughout the training:

4.2 CMOS: the technology under everything

The course derived digital logic from the MOSFET: a voltage-controlled switch. An NMOS transistor conducts when its gate is high; a PMOS conducts when its gate is low. Figure Figure 3 shows the primitive gate of the whole technology, the CMOS inverter, together with the property that makes CMOS dominant: in steady state exactly one of the complementary devices is on, so no direct path exists from VDD to GND and static current is only leakage. The course followed the inverter through its voltage-transfer characteristic and noise margins, then through the transmission gate (a parallel NMOS/PMOS pair used as a bidirectional switch), and showed how latches and edge-triggered flip-flops are assembled from transmission gates and inverters — the same structures that reappear in the static-timing-analysis module of Chapter 7.

4.3 Power, delay and the cost of speed

Power dissipation was split into two families with different remedies:

- [**Static power**] — leakage through supposedly-off transistors (subthreshold leakage, gate leakage). Remedies: power gating with sleep transistors, multi-threshold cells.
- [**Dynamic power**] — charging and discharging of load capacitances at every transition, plus short-circuit and glitching current during switching. Remedy set: clock gating, voltage and frequency scaling, and reducing switching activity.

The governing relation for switching power, used repeatedly in the course and in the capstone design reviews, is

$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f \quad (4.1)$$

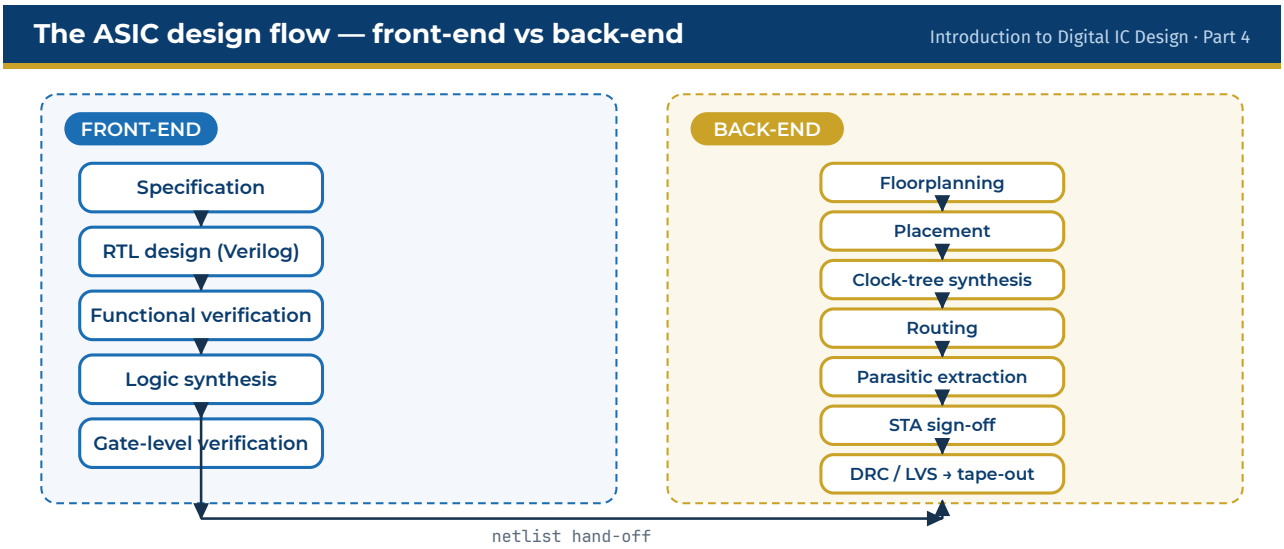


Figure 4: The ASIC flow as taught in the course, split into front-end and back-end. The front-end turns a specification into verified RTL and then into a gate-level netlist through logic synthesis; the back-end turns the netlist into geometry — floorplan, placement, clock-tree synthesis, routing — and closes timing and physical verification before tape-out. The horizontal arrow is the netlist hand-off between the two halves.

with activity factor α, switched capacitance C, supply V and clock frequency f. Because the supply enters squared, voltage scaling is the strongest lever; because frequency and activity enter linearly, clock gating and balanced logic (trees instead of chains, as the RTL course showed for a 4-input XOR) buy power and delay at once. Delay was introduced through propagation delay, and the timing contract of sequential logic through setup, hold, recovery and removal times — defined here physically and revisited analytically in Chapter 7.

4.4 The ASIC design flow

Figure Figure 4 is the course's map of the front-end/back-end flow. Three stages received emphasis because they reappear in every later module: **logic synthesis**, which translates and optimises RTL onto a cell library (and therefore makes coding style a hardware decision); **static timing analysis**, which proves timing closure without simulation (Chapter 7); and **physical verification** (DRC/LVS) before tape-out. The course stressed that incomplete or inaccurate constraints make every downstream step meaningless — a point returned to in the XDC laboratory.

4.5 The FPGA design flow

Figure Figure 5 shows the flow actually executed during the training. An FPGA replaces mask steps with configuration memory: configurable logic blocks, programmable routing and I/O blocks, loaded from a bitstream. The practical difference from the ASIC flow, for a trainee, is iteration speed — hours instead of months — and the absolute centrality of simulation: with no silicon to blame, the waveform is the only witness. All ten laboratories and the capstone of Chapter 9 followed exactly this loop, with the dashed feedback exercised many times.



Figure 5: The FPGA flow used for every deliverable of the training: RTL is simulated against a testbench, synthesised into LUTs and flip-flops, placed and routed inside the fabric, and compiled to a bitstream. The dashed return path is the discipline of the course: a failing waveform sends the design back to RTL, never forward to the board.

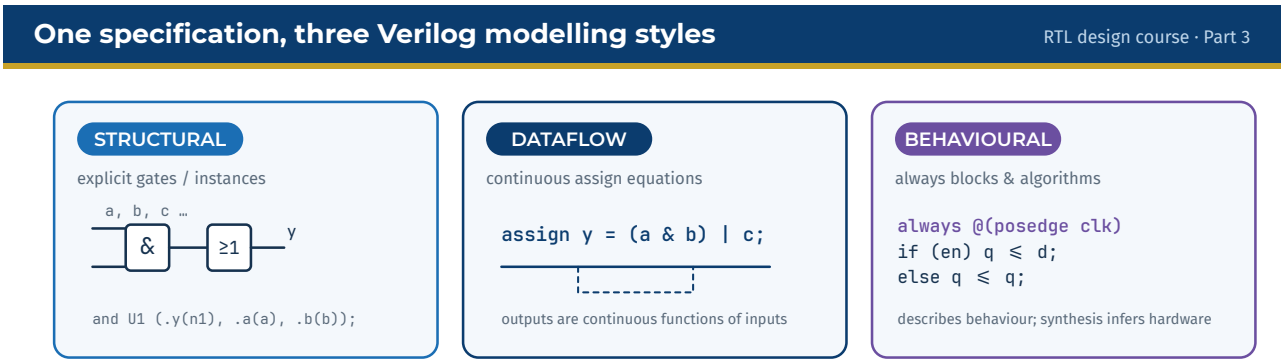


Figure 6: The same specification expressed in the three Verilog modelling styles. **Structural** code instantiates gates and modules explicitly; **dataflow** code writes continuous assignments as Boolean equations; **behavioural** code describes what the circuit does inside procedural blocks and leaves synthesis to infer the structure.

5 RTL design with Verilog

This chapter condenses the *RTL Design Course* and the Verilog-depth lectures of 11–15 July: the language constructs used by every laboratory in Chapter 8, and the semantic rules — assignment semantics, inference rules, procedural constructs — that decide whether a description becomes the intended hardware.

5.1 Abstraction levels and modelling styles

Register-transfer level (RTL) sits between behavioural specification and gate netlists: it describes how data moves between registers and what combinational logic transforms it on the way. The course compared three modelling styles for the same hardware and made the point that style is a hardware decision, not a taste:

Figure Figure 6 contrasts the three styles. Structural descriptions give control and mirror hierarchy; dataflow descriptions are compact for combinational logic; behavioural descriptions scale to algorithms and state machines. The laboratories used all three deliberately — the counter lab of Section 8.7 implements one specification in each style and compares the results.

5.2 Data types: nets, variables, parameters

Verilog's value objects fall into three groups, and confusing the first two is the classic beginner error:

The rule repeated through the labs: a `reg` becomes a flip-flop only when assigned inside an edge-triggered procedural block; assigned inside a level-sensitive block it becomes combinational logic — or an unintended latch if a path is left unassigned.

5.3 Procedural blocks, events and assignment semantics

Two procedural blocks exist: `always`, the synthesizable workhorse that loops forever, and `initial`, a simulation-only construct used by every testbench. Blocks synchronise through event controls (`@(posedge clk)`, `@(*)` / `@*`), and multiple blocks execute concurrently — the language's model of parallel hardware. The single most emphasised rule of the course concerns the two assignment operators:

The sequential-adder example of the course made the danger concrete: mixing the two operators inside one clocked block silently reorders hardware, producing a circuit that simulates like the intent but synthesises differently. All laboratory code in this report follows the golden rule.

Group	Represents	Members and notes
Nets (`wire`)	Physical connections between components	`supply0/1`, `tri/wire`, `tri0/1`, `wand/triand`, `wor/trior`, `trireg`; driven continuously; required for `inout` buses
Variables (`reg`)	Abstract storage in behavioural code	`reg`, `integer`, `real`, `time`; updated by procedural assignment; **not** automatically a flip-flop
Parameters	Compile-time constants	`parameter`, `localparam`, `specparam`; the basis of every parameterised lab (widths, depths, opcodes)

Table 5: The three groups of Verilog value objects, as taught on 11–12 July

Aspect	Blocking =	Non-blocking <=
Execution	Statement completes before the next begins	RHS sampled now, update scheduled at end of time step
Intended hardware	Combinational logic (level-sensitive blocks)	Registers (edge-triggered blocks)
Multiple assignments to one variable	Last write wins, in order	Last scheduled update wins — order-independent
Golden rule	Use in `always @(*)`	Use in `always @(posedge clk)`; never mix in one block

Table 6: Blocking versus non-blocking assignment — the rule set drilled in the labs

Variant	Treatment of x / z in items	Guidance from the course
`case`	Exact match; x and z are compared literally	Default for RTL and for checking real values
`casex`	x, z and `?` are wildcards in both expression and items	Concise for encoders/decoders; hides initialisation problems — not recommended
`casez`	Only z and `?` are wildcards; x still matches exactly	Preferred wildcard form; safer for uninitialised RTL; useful in testbenches

Table 7: Behaviour of the case variants with respect to x and z bits

5.4 Conditional constructs: `if`, `case`, `casex`, `casez`

The `case` family received dedicated attention on 13 July because the three variants differ in how they treat unknown (`x`) and high-impedance (`z`) bits:

5.5 Functions and tasks

The 13–14 July modules separated the two reusable procedural constructs, and the parity lab (Section 8.9) put functions to work inside synthesizable code:

A take-home exercise asked for the failure mode of static functions under re-entrant calls; the answer — a second overlapping invocation overwrites the single static copy of the locals, which `automatic` storage prevents — was demonstrated with a fork/join testbench.

5.6 Operators, literals and synthesis-aware coding

The course catalogued Verilog's operator families (arithmetic, logical, bitwise, reduction, shift, equality, conditional), its sized literal syntax (a width, a base letter and the digits, e.g. `8'hFF` or `4'b1010`), concatenation and replication. Two synthesis lessons accompanied them. First, the same function can synthesise to different hardware: three 4-bit adders written with different constraints produced different adder architectures. Second, structure matters even for pure combinational logic — a 4-input XOR built as a balanced tree of 2-input gates beats a chain in delay, at a small area cost, and a single 4-input cell beats both in area. Power entered here through glitching: unbalanced chains glitch more, and glitches are pure wasted energy (Equation 4.1).

Aspect	Function	Task
Simulation time	Never — no `#`, `@`, `wait`	May consume time freely
Returns	Exactly one value, via its own name	Zero or many, via `output`/`inout` arguments
May call	Functions only	Tasks and functions
Synthesizable	Yes, when purely combinational	Generally no — testbench stimulus and checkers
Storage	`static` by default; `automatic` gives per-call frames	Same rule; `task automatic` required for re-entrant calls

Table 8: Functions versus tasks

5.7 Sequential building blocks

Latches were presented as level-sensitive transparency — useful inside CMOS cells, dangerous in RTL because an incomplete `if` or `case` infers one silently; the course's remedy is complete assignments in every level-sensitive block. Edge-triggered D flip-flops then provide the state of the machine, with synchronous versus asynchronous reset as a design choice (asynchronous acts immediately, synchronous keeps reset inside the clock domain). Counters, shift registers and RAMs (modelled as two-dimensional arrays) completed the vocabulary; all appear as laboratories in Chapter 8.

5.8 Testbenches and verification discipline

The final part of the course treated verification as a first-class activity. A testbench instantiates the design under test (DUT), generates clock and reset, applies stimulus — fixed, looped or randomised with `$random` — and observes results through `$display`/`$monitor`/`$strobe` or, better, through self-checking tasks that compare against expected values and report `TEST PASSED`/`TEST FAILED`. Waveform dumping (`$dumpfile`/`$dumpvars`) makes every run reviewable. The verification loop of Figure Figure 5 is exactly this discipline, and every repository testbench in Chapter 8 is self-checking.

6 Finite-state machines

The FSM module (16 and 19 July) supplied the design technique behind the sequence-detector lab, the debouncer, the capstone controllers and — conceptually — every controller written during the training.

6.1 What an FSM is and why it matters

A finite-state machine models a system that transits among a finite number of internal states, with each transition determined by the current state and the external inputs. Unlike regular sequential circuits (counters, shifters) whose next-state logic is structured, an FSM's next-state logic is built from scratch — "random" logic — which is precisely why it can implement arbitrary control policies. In practice the FSM is the **controller** of a digital system: it examines commands and status flags and asserts the control signals that steer a datapath of regular components. Every lab controller in Chapter 8 — the VeriRISC control unit, the PISO sequencer, the capstone ECUs — is an instance of this pattern.

6.2 Moore versus Mealy

The lecture's worked pair — a Moore traffic-light controller and a Mealy sequence detector — made the trade-off tangible, and the laboratory of Section 8.8 implements both detector flavours so the difference can be read on a waveform.

6.3 Representation: state diagrams and ASM charts

An FSM is specified graphically before it is coded. State diagrams draw states as annotated circles and transitions as arcs labelled with the input condition that fires them; Moore outputs live inside the circle, Mealy outputs on the arcs. Algorithmic-state-machine (ASM) charts pack the same information into a flowchart-like form — one state box per state with a single exit, decision boxes testing conditions, conditional output boxes for Mealy outputs — which scales better when transition conditions grow complex. The detector of Figure Figure 19 is drawn as a state diagram because its conditions are single bits.

Feature	Moore	Mealy
Output depends on	Current state only	Current state and inputs
Output timing	Changes only with state (clock edges)	May change immediately with inputs
States needed	Usually more	Typically fewer
Glitch susceptibility	Lower — outputs stable between edges	Higher — outputs react to inputs
Reaction speed	One cycle later	Same cycle
Typical use	Control FSMs where stability matters	Protocol/datapath FSMs needing fast reaction

Table 9: Moore and Mealy machines compared, as tabulated in the FSM lecture

Encoding	Flip-flops for N states	Strengths	Weaknesses / use
Binary	ceil(log2(N))	Minimal area	Deeper decode logic; small FSMs
One-hot	N	Shallow, fast next-state logic; easy debug; FPGA-friendly	Many flip-flops; power in large FSMs
Gray	ceil(log2(N))	One bit changes per transition: low switching power	Only helps sequential traversal

Table 10: State-encoding techniques and their trade-offs

6.4 Coding an FSM in Verilog

The lecture's development procedure mirrors that of any sequential circuit — separate the state register, then derive next-state and output logic — with five code segments: state encoding, register/signal declaration, the state register itself, the next-state logic following the diagram, and the output logic. The recommended idiom is three procedural blocks: a clocked block for the state register, a level-sensitive block for the next state, and (for Moore machines) a third block or assignment for outputs.

On FPGAs the abundant register file makes one-hot encoding usually the best trade, while binary encoding protects area in ASIC designs — a decision the capstone controllers resolve explicitly in their design notes.

7 Static timing analysis

The closing technical module (22 July, material co-authored by Eng. Mohammed Salah Aboshosha and Eng. Ahmed Abdelazeem) taught static timing analysis (STA): proving that a circuit meets its timing constraints **without simulation**. This chapter follows the module's own order: why STA exists, the vocabulary of arcs and unateness, the device-level origin of setup and hold, and finally paths, slack and constraints — closed by the XDC laboratory.

7.1 Why STA

- [**It verifies timing.**] It checks timing between synchronous clocks; it does not verify functionality — that remains simulation's job.
- [**It is exhaustive.**] Formal, mathematical path analysis replaces vectors: every path is examined, not only the stimulated ones.
- [**It is fast.**] Orders of magnitude faster than gate-level simulation, and further still from transistor-level simulation.
- [**It is path-based and constraint-driven.**] STA identifies timing paths and computes the arcs inside them; a path without a constraint is, by default, not reported at all — so incomplete constraints silently blind the analysis.

7.2 Timing arcs and unateness

A **timing arc** is a path from each input to each output of a cell. Combinational cells have an arc per input-output pair; a flip-flop has a delay arc from clock to Q and constraint arcs (setup/hold checks) from clock to D; a latch has a clock-to-Q arc and a transparent D-to-Q arc. **Unateness** classifies how an output responds to an input transition:

Unateness lets the analyser propagate only the transitions that matter, which is a large part of why STA is fast.

Class	Rule	Example cells
Positive unate	Output transition follows the input transition (rise→rise, fall→fall)	AND, OR, buffer
Negative unate	Output transition inverts the input transition (rise→fall, fall→rise)	NAND, NOR, inverter
Non-unate	Output direction cannot be deduced from the input direction alone; other inputs decide	XOR, XNOR, mux select

Table 11: Unateness classes with the lecture's cell examples

Setup and hold windows around the capturing clock edge STA · timing parameters

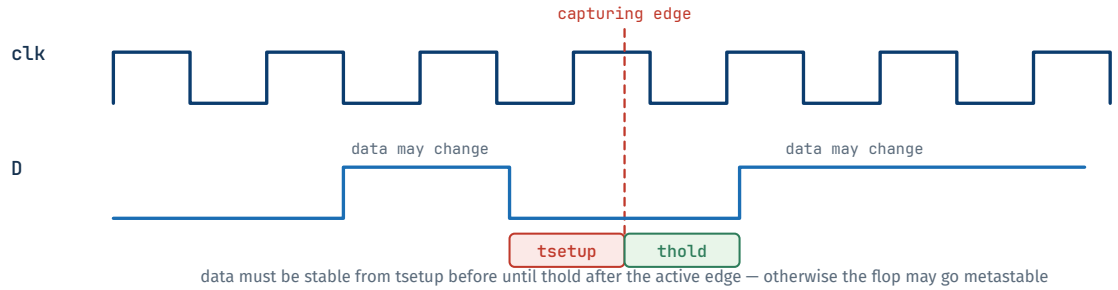


Figure 7: The setup and hold windows. The dashed vertical line is the capturing clock edge; the data waveform must stop changing t_setup before that edge and may resume only t_hold after it. Data transitions inside either window risk metastability at the capturing flop.

7.3 From transistors to setup and hold

The module rebuilt storage from devices upward: the transmission gate (parallel PMOS/NMOS with complementary gates, giving an almost constant on-resistance bidirectional switch); the level-sensitive D latch built from two transmission gates and two inverters, transparent while its clock level is active; and the edge-triggered D flip-flop obtained by cascading a negative and a positive latch, so data moves only at the active edge. From that circuitry follow the three parameters used everywhere afterwards: t_cq (clock-to-Q delay, different for rising and falling outputs), t_setup (data must arrive this early **before** the edge) and t_hold (data must remain stable this long **after** the edge). Figure Figure 7 draws the two windows around a capturing edge; violating either means the capturing latch may resolve to a metastable level between logic 0 and 1.

The module then introduced the **launch** and **capture** edges: the launch edge clocks data out of the source flop, the capture edge samples it at the destination flop one period later. All constraint arithmetic of the next section is bookkeeping between these two edges.

7.4 Paths, constraints and slack

A **timing path** chains every arc from a start point to an end point; the module distinguishes input→register, register→register, register→output and pure combinational input→output paths (chips in Figure Figure 8). Path delay is simply the sum of cell delays (from the library's delay model) and net delays (from layout parasitics or a wire-load model). Against that arrival time, the constraint defines a required time, and their difference is **slack** — the currency of timing closure:

$$\textbf{slack_setup} = T_{required} - T_{arrival}\ ,\ \textbf{slack_hold} = T_{arrival} - T_{required} \qquad (7.1)$$

Register-to-register timing path and the four path classes Static timing analysis

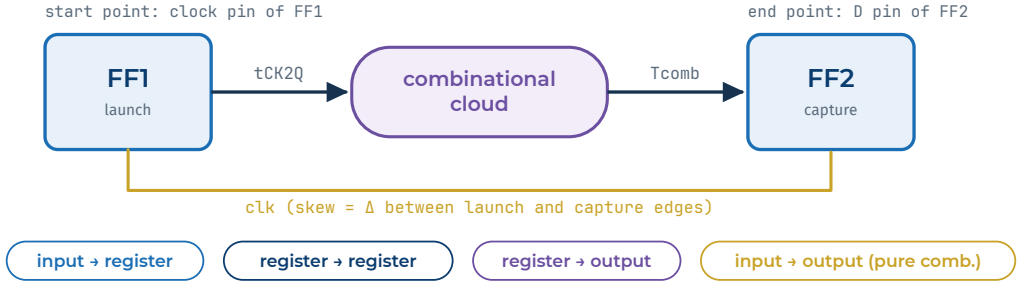


Figure 8: A register-to-register timing path: launch flip-flop, combinational cloud, capture flip-flop, with the shared clock underneath. The start point of a path is an input port or a clock pin of a sequential cell; the end point is an output port or a data pin of a sequential cell. The four path classes taught in the module are listed below.

For the register-to-register path of Figure Figure 8 the module derived the two checks that every later lab and the capstone were measured against:

$$T_{clk} + T_{skew} \geq T_{c2q} + T_{comb} + T_{setup} \quad \text{(setup / max path)} \qquad (7.2)$$

$$T_{c2q} + T_{comb} \geq T_{hold} + T_{skew} \quad \text{(hold / min path)} \qquad (7.3)$$

with direct analogues for input-to-register paths (input delay models the outside launch logic as an equivalent t_c2q plus combinational delay) and register-to-output paths (output delay models the outside capture logic as an equivalent setup requirement). Negative slack on a max path means the clock period is unachievable; negative slack on a min path means hold violations, which — unlike setup violations — cannot be fixed by slowing the clock.

7.5 Constraint writing laboratory (XDC)

The module's laboratory applied the theory to a small design, `timing_paths.v`, containing one register-to-register path, one input-to-register path, one register-to-output path and one pure combinational path — deliberately one of each class. Constraints were written in XDC for Vivado, parameterised as percentages of a 10 ns clock period so that every delay budget is derived, not guessed:

timing_paths.xdc (excerpt) FPGA-labs-project/22_7

```
#create_clock -name sys_clk -period 10.0 [get_ports clk]
#create_clock -name v_clk -period 10.0

#set_clock_uncertainty 0.5 [get_clocks sys_clk]
#set_clock_uncertainty 0.5 [get_clocks v_clk]

#set_input_delay -clock sys_clk -max 2.5 [get_ports in_data]
#set_input_delay -clock sys_clk -min 0.5 [get_ports in_data]

#set_output_delay -clock sys_clk -max 3.0 [get_ports out_data]
#set_output_delay -clock sys_clk -min 1.0 [get_ports out_data]

#set_max_delay -from [get_ports in_comb] -to [get_ports out_comb] 8.0
#set_min_delay -from [get_ports in_comb] -to [get_ports out_comb] 2.0

# ============================================================
#
1. Base Clock Period & Percentage Parameters


# ============================================================
set CLK_PERIOD 10.0

set PCT_UNCERTAINTY 0.05 ;
set PCT_INPUT_MAX 0.25 ;
set PCT_INPUT_MIN 0.05 ;
set PCT_OUTPUT_MAX 0.30 ;
set PCT_OUTPUT_MIN 0.10 ;
set PCT_MAX_DELAY 0.80 ;
set PCT_MIN_DELAY 0.20 ;

# ============================================================
#
2. Calculated Delay Values


# ============================================================
set clk_uncertainty [expr {$CLK_PERIOD * $PCT_UNCERTAINTY}]
set input_delay_max [expr {$CLK_PERIOD * $PCT_INPUT_MAX}]
set input_delay_min [expr {$CLK_PERIOD * $PCT_INPUT_MIN}]
set output_delay_max [expr {$CLK_PERIOD * $PCT_OUTPUT_MAX}]
set output_delay_min [expr {$CLK_PERIOD * $PCT_OUTPUT_MIN}]
set comb_max_delay [expr {$CLK_PERIOD * $PCT_MAX_DELAY}]
set comb_min_delay [expr {$CLK_PERIOD * $PCT_MIN_DELAY}]

# ============================================================
#
3. Clock Definitions & Uncertainties


# ============================================================
create_clock -name sys_clk -period $CLK_PERIOD [get_ports clk]
create_clock -name v_clk -period $CLK_PERIOD
```

```
set_clock_uncertainty $clk_uncertainty [get_clocks sys_clk]
set_clock_uncertainty $clk_uncertainty [get_clocks v_clk]

# ============================================================
#
4. Input Delays (IN2REG)


# ============================================================
set_input_delay -clock sys_clk -max $input_delay_max [get_ports in_data]
set_input_delay -clock sys_clk -min $input_delay_min [get_ports in_data]

set_input_delay -clock v_clk -max $input_delay_max [get_ports in_comb]
set_input_delay -clock v_clk -min $input_delay_min [get_ports in_comb]

# ============================================================
#
5. Output Delays (REG2OUT)


# ============================================================
set_output_delay -clock sys_clk -max $output_delay_max [get_ports out_data]
set_output_delay -clock sys_clk -min $output_delay_min [get_ports out_data]

set_output_delay -clock v_clk -max $output_delay_max [get_ports out_comb]
set_output_delay -clock v_clk -min $output_delay_min [get_ports out_comb]

# ============================================================
#
6. Max / Min Delays (IN2OUT)


# ============================================================
set_max_delay -from [get_ports in_comb] -to [get_ports out_comb] $comb_max_delay
set_min_delay -from [get_ports in_comb] -to [get_ports out_comb] $comb_min_delay
```

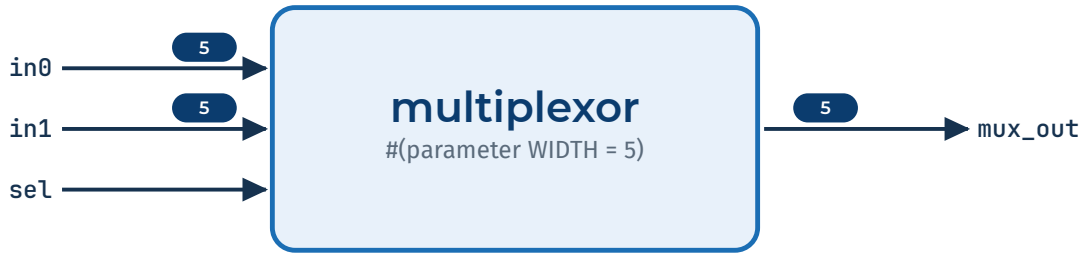
The exercise closed the loop of Section 4.4: the same four commands (`create_clock`, `set_input_delay`, `set_output_delay`, `set_max_delay`/`set_min_delay`) that make STA meaningful in an ASIC sign-off are what make synthesis optimise the right paths on an FPGA. Reviewing the reported slack of each of the four paths against Equations 7.1–7.3 was the final laboratory act of the technical programme.

8 Hands-on laboratories: what I designed and verified

This chapter is the core of the report: the ten laboratory deliverables of the training, each presented as it was worked — specification, design, code, simulation evidence and the lesson taken. All sources, testbenches and ModelSim projects live in the repository `Zeyad1Khaled/NTI_Labs` (Appendix A); every testbench is self-checking and prints `TEST PASSED` on success. Waveform extracts in Section 8.4 are real ModelSim VCD dumps from that repository.

#	Laboratory	Key concepts exercised	Repository project
1	Parameterised 2:1 multiplexer	modules, parameters, level-sensitive blocks	`multiplexer-2-to-1`
2	Tri-state bus driver	high-impedance `z`, replication, buses	`tri-state-driver`
3	8-bit ALU, eight opcodes	operators, `case`, reduction equality	`arithmetic-logic-unit`
4	Serial datapath RAM–PISO–SIPO–ALU	hierarchy, handshaking, serial links	`serial-datapath`
5	Bidirectional-port memory	`inout` nets, read/write arbitration	`bidirectional-memory`
6	Generic register	non-blocking assignment, synchronous reset	`microcontroller-register`
7	VeriRISC control unit	`case` over phases, combinational control	`microcontroller-controller`
8	2-bit up/down counter, three styles	behavioural vs structural vs gate-level	`up-down-counter`
9	110101 sequence detector (Moore)	FSM states, overlapping detection	`sequence-detector-110101`
10	Stream parity generator	functions inside synthesizable code	`stream-parity-generator`

Table 12: Laboratory programme at a glance



sel = 0 → in0 passes · sel = 1 → in1 passes · WIDTH parameterised (default 5)

Figure 9: Block diagram of Laboratory 1 as drawn on the lab sheet: two 5-bit data inputs, a 1-bit select and a 5-bit output; the width is a module parameter, so the same source serves any address width.

8.1 Laboratory 1 — Parameterised 2:1 multiplexer

Specification. The address multiplexer of a small CPU selects the instruction address during fetch and the operand address during execute. Its width is parameterised (default 5); `sel = 0` passes `in0`, `sel = 1` passes `in1`.

multiplexer-2-to-1/src/mux.v Zeyad1Khaled/NTI_Labs

```
module multiplexor #(parameter WIDTH)(
 input wire [WIDTH-1:0] in0,in1,
 input sel,
  output reg [WIDTH-1:0] mux_out
);

always@(*)begin
  if(sel)begin
    mux_out=in1;
  end
  else
    mux_out=in0;
end

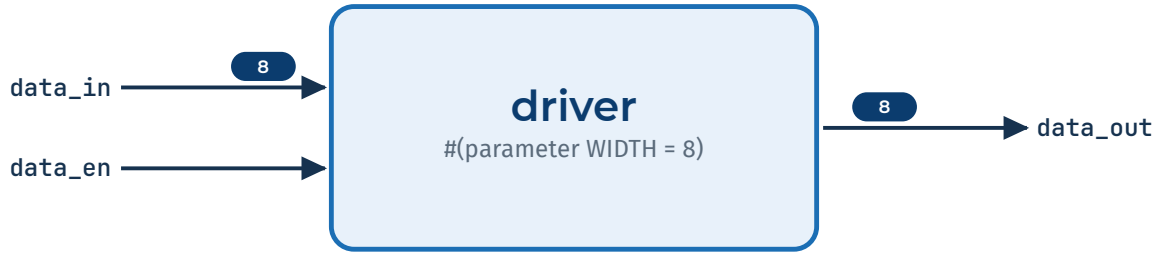
endmodule
```

Result and lesson. The design passes its self-checking testbench for both select values and for a re-parameterised width of 12. The lesson is the first of the course: a level-sensitive `always @(*)` block with a complete if/else synthesises to a multiplexer — the `if` **is** hardware selection, and the parameter makes the hardware scalable without a second source file.

8.2 Laboratory 2 — Tri-state bus driver

Specification. An 8-bit (parameterised) driver presents `data_in` on `data_out` while `data_en` is true and releases the bus to high impedance otherwise — the construct that lets several devices share wires.

tri-state-driver/src/driver.v Zeyad1Khaled/NTI_Labs



data_en = 1 → data_out = data_in · data_en = 0 → data_out = 8'bz (high impedance)

Figure 10: Laboratory 2 block diagram. When disabled the output is not a value but an absence of one: the replicated literal `{WIDTH{1'bz}}` drives every bit of the bus to `z`.



eight opcodes: HLT SKZ ADD AND XOR LDA STO JMP · a_is_zero = (in_a == 0)

Figure 11: Laboratory 3 block diagram: two 8-bit operands and a 3-bit opcode enter; an 8-bit result and the zero flag leave. The zero flag is asynchronous — pure combinational reduction equality on `in_a`.

```
module driver #(parameter WIDTH)(
  input [WIDTH-1:0] data_in,
  input data_en,
  output reg [WIDTH-1:0] data_out
);
always@(*)begin
  if(data_en)
    data_out=data_in;
else
  data_out={WIDTH{1'bz}};

end
endmodule
```

Result and lesson. The testbench verifies the driven values and, with a second driver on the same wire, that a disabled driver truly releases the bus. The lesson is the net/variable boundary of Section 5.2: only a net (`wire`) can be high-impedance or multi-driven, and the replication idiom `{WIDTH{1'bz}}` is the portable way to release a parameterised bus.

8.3 Laboratory 3 — 8-bit ALU with eight opcodes

Specification. The ALU executes the eight VeriRISC instructions on 8-bit operands, selected by a 3-bit opcode, and reports asynchronously whether operand A is zero.

arithmetic-logic-unit/src/alu.v Zeyad1Khaled/NTI_Labs

```
module alu #(parameter WIDTH=8)(
  input [WIDTH-1:0]in_a,in_b,
  input [2:0]opcode,
  output reg a_is_zero,
  output reg [WIDTH-1:0] alu_out

);
always @(*)begin
  if(in_a)
```

Instruction	Opcode	Operation	Result
HLT	000	pass A	`in_a => alu_out`
SKZ	001	pass A (skip if zero elsewhere)	`in_a => alu_out`
ADD	010	addition	`in_a + in_b => alu_out`
AND	011	bitwise AND	`in_a & in_b => alu_out`
XOR	100	bitwise XOR	`in_a ^ in_b => alu_out`
LDA	101	pass B	`in_b => alu_out`
STO	110	pass A	`in_a => alu_out`
JMP	111	pass A	`in_a => alu_out`

Table 13: Opcode encoding and operations of the laboratory ALU

```
    a_is_zero=1'b0;
else
  a_is_zero=1'b1;

end

always@(*)begin
  case(opcode)
    3'b000:alu_out<=in_a;
    3'b001:alu_out<=in_a;
    3'b010:alu_out<=in_a+in_b;
    3'b011:alu_out<=in_a&in_b;
    3'b100:alu_out<=in_a^in_b;
    3'b101:alu_out<=in_b;
    3'b110:alu_out<=in_a;
    3'b111:alu_out<=in_a;


  endcase

end

endmodule
```

Result and lesson. All eight opcodes plus the zero flag are checked exhaustively on directed vectors and randomised operands. The lesson is operator-level synthesis: one `case` over the opcode becomes an 8:1 multiplexer of operator outputs, and the asynchronous zero flag (`a_is_zero = (in_a == 0)`) shows that a reduction equality is real comparison hardware, not code.

8.4 Laboratory 4 — Integrated serial datapath (RAM → PISO → SIPO → ALU)

Specification. The capstone laboratory of the RTL module: instruction words stored in a RAM are streamed **bit-serially** over a single wire into a shift register that re-assembles them, and the completed word is executed by the ALU of Laboratory 3 — one integrated `TOP_MODULE`.

serial-datapath/src/top_module.v Zeyad1Khaled/NTI_Labs

```
`timescale 1ns/1ps
module TOP_MODULE #(parameter WIDTH=20,Addr_width=8,ALU_OUT=8)(
  input clk,rst_n,wr_en,
  input [Addr_width-1:0] addr,
  input [WIDTH-1:0] din,

  output [ALU_OUT-1:0] out,
  output a_is_zero
);
//ALU Wire
wire [WIDTH-1:0] parallel_out;
//RAM Wire
wire [WIDTH-1:0] ram_dout;
wire ram_valid;
```

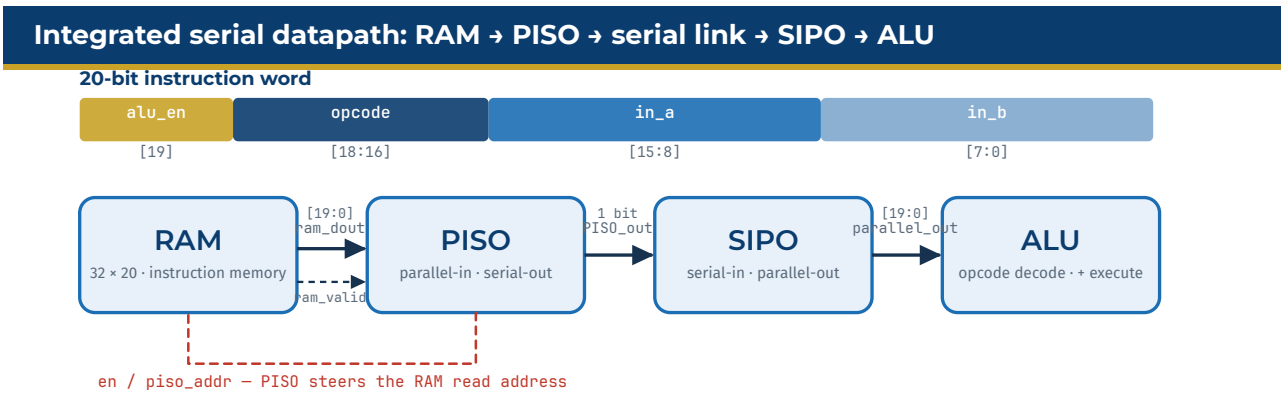


Figure 12: The integrated datapath. The RAM holds 20-bit words; the PISO serialises a word under its own handshake (`en`, `ram_valid`) and even steers the RAM read address (`piso_addr`); the SIPO re-assembles the stream into `parallel_out`; the ALU decodes bit 19 as enable, bits 18:16 as opcode and the two bytes as operands, exactly as the word-format band on top shows.

```
//PISO wire
wire PISO_out;
wire en;
wire PISO_valid;
wire [Addr_width-1:0] piso_addr;

RAM U1(
.clk(clk),
.rst_n(rst_n),
.wr_en(wr_en),
.addr(wr_en ? addr : piso_addr),
.din(din),
.rd_en(en),
.dout(ram_dout),
.valid(ram_valid));

PISO U2 (
.clk(clk),
.parallel_in(ram_dout),
.rst_n(rst_n),
.ram_valid(ram_valid),
.serial_out(PISO_out),
.en(en),
.valid(PISO_valid),
.addr(piso_addr));

SIPO U3(
.clk(clk),
.rst_n(rst_n),
.shift_en(PISO_valid),
.serial_in(PISO_out),
.parallel_out(parallel_out));

ALU U4 (
.alu_en(parallel_out[19]),
.opcode(parallel_out[18:16]),
.in_a(parallel_out[15:8]),
.in_b(parallel_out[7:0]),
.alu_out(out),
.a_is_zero(a_is_zero)
);
endmodule
```

The design is four modules and a handful of handshakes, and it is where the training's concepts first had to work **together**: hierarchy and named port connection, two clocked state machines (PISO bit counter, SIPO shift counter), a parameterised RAM, and the ALU — with `en`/`valid` pairs playing the role of cheap readiness protocols.

Result and lesson. Figures Figure 13 and Figure 14 are actual simulation dumps: the fetch burst, the assembled word and the per-word results all match the testbench log (`OUT`, `ZERO`, `PISO addr` printed per pass). The lesson is systemic: serialisation buys wires at the price of latency and protocol discipline, and

Simulation extract 1 — bit-serial fetch of one 20-bit instruction word

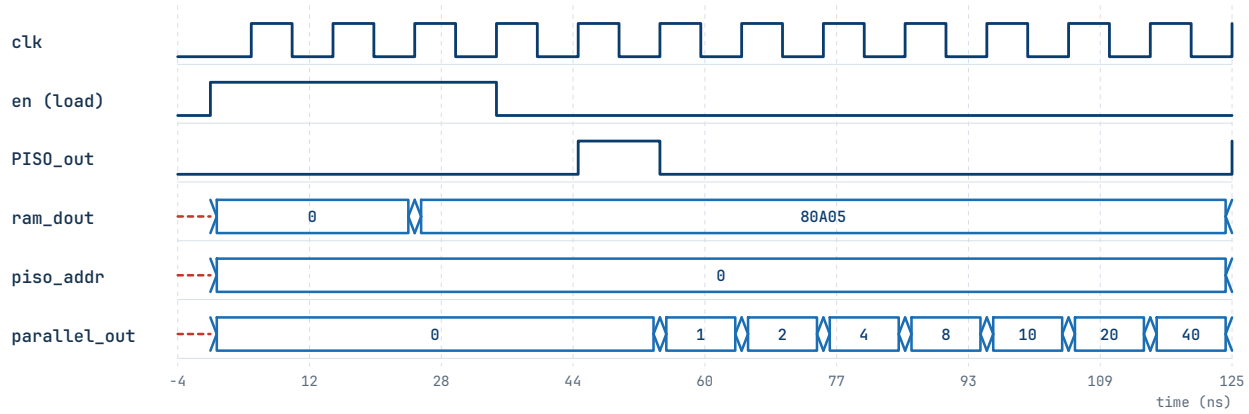


Figure 13: ModelSim extract of one serial fetch (repository VCD dump). While `en` is high the PISO loads `ram_dout` (word `0x80302`: enable, opcode ADD, operands 3 and 2) and then clocks it out on `PISO_out` one bit per cycle; `parallel_out` visibly assembles in the SIPO shift register, bit by bit, until the full word stands at the end of the burst.

Simulation extract 2 — instruction words executed back-to-back

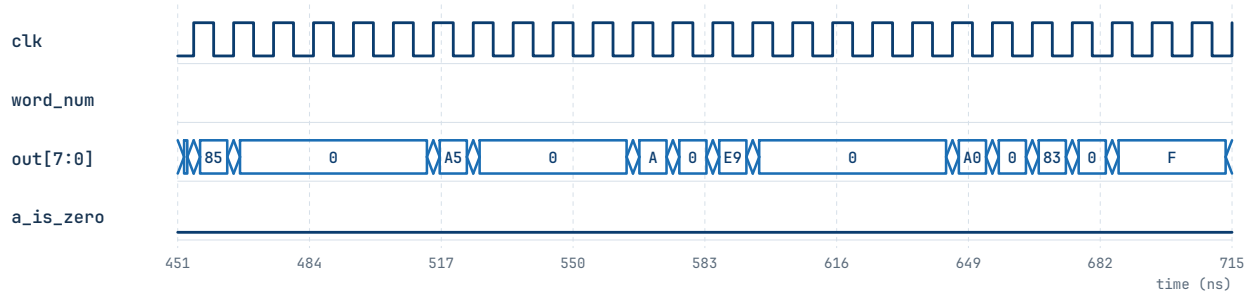


Figure 14: ModelSim extract of consecutive executions from the same dump. `word_num` advances as each word completes; `out[7:0]` updates with each ALU execution, and `a_is_zero` pulses exactly on the words whose result is zero — the self-checking testbench's expectations reproduced on silicon-time waveforms.

a handshake signal that is almost right produces a datapath that is wrong once per thousand words — which is exactly what the first debugging session taught.

8.5 Laboratory 5 — Single-bidirectional-port memory

Specification. A 32 × 8 memory with one bidirectional data port: write on the rising clock edge when `wr` is asserted; drive the bus combinationally when `rd` is asserted; never both at once.

bidirectional-memory/src/memory.v — Zeyad1Khaled/NTI_Labs

```
module memory #(parameter AWIDTH=5,DWIDTH=8)(
  input wire wr,rd,
  input [AWIDTH-1:0] addr,
  input clk,

  inout [DWIDTH-1:0] data
);
reg [DWIDTH-1:0] memory [0:(1<<(AWIDTH))-1];

always@(posedge clk)begin

  if(wr && !rd)begin
    memory[addr]<=data;
  end

end
assign data=rd?memory[addr]:{DWIDTH{1'bz}};
endmodule
```

Result and lesson. The testbench writes a pattern, reads it back, and probes the bus in the idle state to confirm `z`. Two lessons: `inout` ports force the net side of the type system, and a bidirectional bus makes arbitration a design obligation — the `wr && !rd` guard and the conditional drive are the whole protocol.

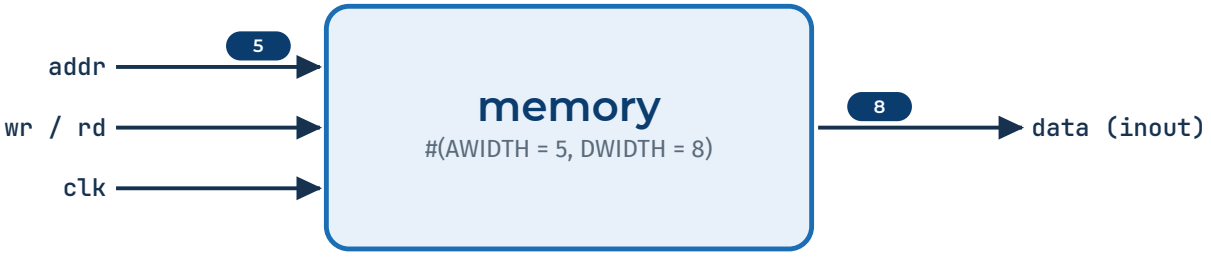


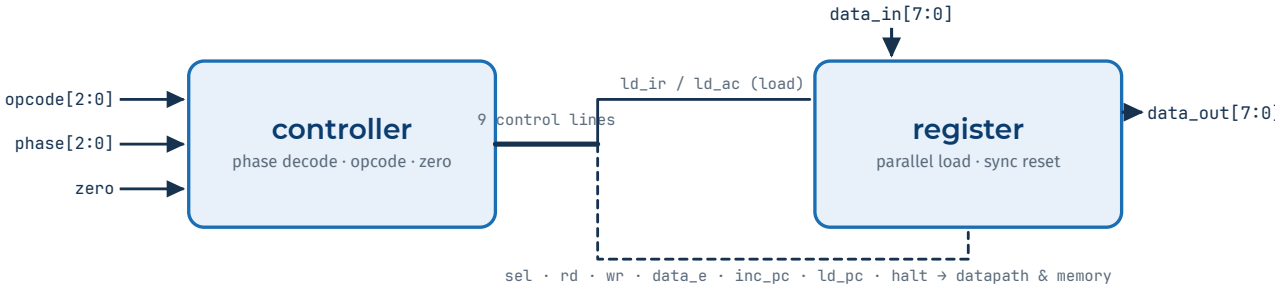
Figure 15: Laboratory 5 block diagram. The single `data` port is an `inout` net: an input during writes, an output during reads, and high impedance otherwise — the electrical reason the port must be a net rather than a variable.



parallel-load register · synchronous active-high reset · serves as IR and accumulator

Figure 16: The generic register of Laboratory 6: clocked load with synchronous reset — the canonical non-blocking-assignment storage element.

VeriRISC-style control unit and register file Labs 6 & 7



case (phase) decodes the fetch/execute phases; combinational conditions on opcode and zero drive every control line.

Figure 17: How the two laboratories compose: the controller's nine control lines are pure combinational functions of `opcode`, `phase` and `zero`; the load lines (`ld_ir`, `ld_ac`) clock data into register instances, while the remaining lines steer the datapath and memory around them.

8.6 Laboratory 6 — Generic register and Laboratory 7 — VeriRISC control unit

Specification. One register definition serves as instruction register and accumulator: 8-bit parallel load under `load`, synchronous active-high reset. The control unit decodes opcode, fetch/execute phase and the zero flag into nine control signals.

microcontroller-register/src/register.v Zeyad1Khaled/NTI_Labs

```
module register #(parameter WIDTH=8)(
  input [WIDTH-1:0] data_in,
  input load,
  input clk,rst,

  output reg [WIDTH-1:0] data_out

);
always@(posedge clk)begin
  if(rst)begin
    data_out<={WIDTH{1'b0}};
  end
  else if (load)begin
    data_out<=data_in;
  end
end

endmodule
```

microcontroller-controller/src/controller.v Zeyad1Khaled/NTI_Labs

```
module controller(
```

```
    input [2:0] opcode ,
    input [2:0] phase,
    input zero,

    output reg sel,
    output reg rd,ld_ir,inc_pc ,halt,
    output reg ld_pc,data_e,ld_ac,wr
  );
    always @(*) begin

        case (phase)
            3'b000:begin
                {rd,ld_ir,halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
                sel=1;
            end
            3'b001:begin
                {ld_ir,halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
                sel=1; rd=1;
            end
            3'b010:begin
                sel=1;
                rd=1;
                ld_ir=1;
                {halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
            end
            3'b011:begin
                sel=1;
                rd=1;
                ld_ir=1;
                {halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
            end
            3'b100:begin
                inc_pc=1;
                halt=(opcode==0);
                {sel,rd,ld_ir,ld_ac,ld_pc,wr,data_e}='b0;
            end
            3'b101:begin
                rd=(opcode>=2&&opcode<=5);
                {sel,ld_ir,halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
            end
            3'b110:begin
                rd=(opcode>=2&&opcode<=5);
                inc_pc=(opcode==1&&zero);
                ld_pc=(opcode==7);
                data_e=(opcode==6);
                {sel,ld_ir,halt,ld_ac,wr}='b0;
            end
            3'b111:begin
                rd=(opcode>=2&&opcode<=5);
                ld_ac=(opcode>=2&&opcode<=5);
                ld_pc=(opcode==7);
                wr=(opcode==6);
                data_e=(opcode==6);
                {sel,ld_ir,inc_pc,halt}='b0;
            end
            default:{sel,rd,ld_ir,halt,inc_pc,ld_ac,ld_pc,wr,data_e}='b0;
        endcase
        end


endmodule
```

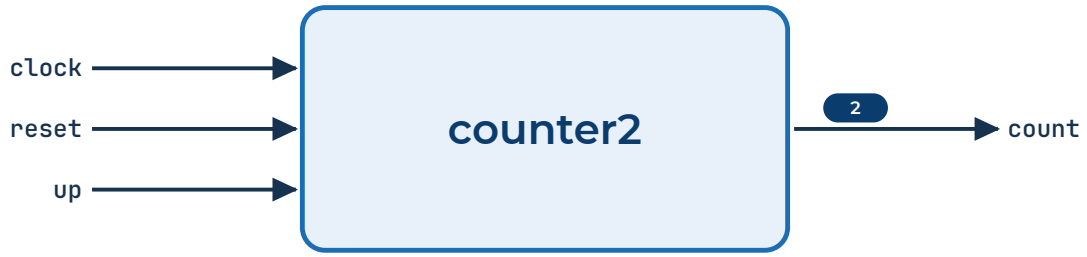
Result and lesson. The controller testbench walks every opcode through every phase and compares all nine control lines against the specification table — the most complete self-check written during the training. The lesson is the controller/datapath split of Chapter 6: a `case` over phases **is** a state machine, and writing the control table first made the RTL almost mechanical.

8.7 Laboratory 8 — 2-bit up/down counter in three modelling styles

Specification. One circuit, three descriptions: behavioural (one procedural block), gate-level (hand-derived next-state equations on primitives), and structural (instantiated D flip-flops and gates), plus a shared testbench and a function/task-based variant.

up-down-counter/src/behavioral.v Zeyad1Khaled/NTI_Labs

```
`timescale 1ns/1ps
module counter2_behavioral (
    input clock,
```



2-bit up/down counter · implemented behaviourally, structurally and at gate level

Figure 18: The counter interface shared by all three implementations: clock, asynchronous active-high reset, direction input and a 2-bit count. Identical ports are what make the three styles comparable under one testbench.

```
    input reset, // asynchronous, active-high
    input up, // 1 = count up, 0 = count down
    output reg [1:0] count
);

always@(posedge clock or posedge reset)begin
  if(reset)begin
    count<='b0;
  end
else if(up)begin
  count<=count+1;
end
else
  count<=count-1;


end
endmodule
```

Result and lesson. All three netlists count up, count down and reset identically; the behavioural source is eight lines where the gate-level source is thirty. The lesson is abstraction as productivity: identical behaviour, radically different maintenance cost — and the gate-level detour is what made the synthesised result of the behavioural version legible afterwards.

8.8 Laboratory 9 — Overlapping 110101 sequence detector

Specification. Detect the serial pattern 110101 on `din`, in both overlapping and non-overlapping flavours, as a clocked Moore machine.

Moore state diagram — overlapping detector for the sequence 110101

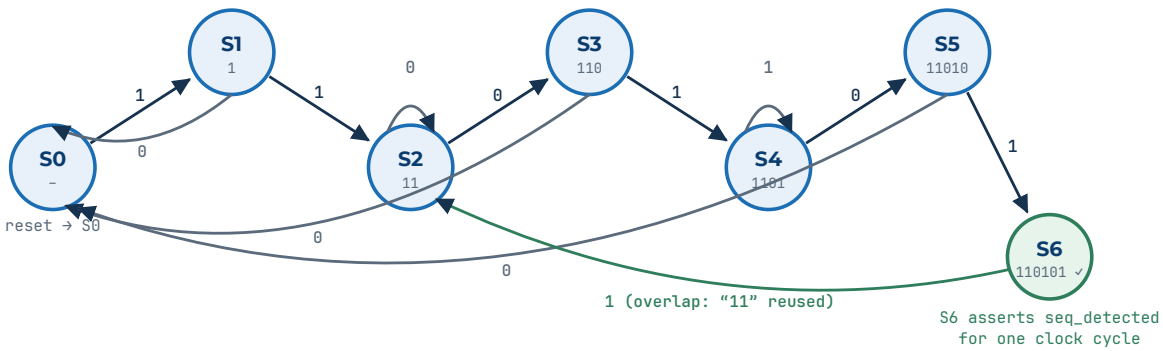


Figure 19: Moore state diagram of the overlapping detector. Each state records how much of the target pattern has been matched; a mismatch falls back to the longest state that is still a valid prefix (grey arcs), which is precisely what makes detection overlapping. State S6 asserts `seq_detected` for one clock cycle and, on a further `1`, re-enters S2 because the suffix “11” is already a prefix of the next match.

sequence-detector-110101/src/overlapping.v Zeyad1Khaled/NTI_Labs

```
module overlapping(
  input din,
  input clk,
  input rst,

  output reg seq_detected
);
reg [2:0]current_state,next_state;

localparam
s0='b0,
s1='b01,
s2='b10,
s3='b11,
s4='b100,
s5='b101,
s6='b110;

always@(*)begin
  seq_detected=0;
  case(current_state)
    s0:next_state=din?s1:s0; //1
    s1:next_state=din?s2:s0; //11
    s2:next_state=din?s2:s3; //110
    s3:next_state=din?s4:s0; //1101
    s4:next_state=din?s2:s5; //11010
    s5:next_state=din?s6:s0; //110101
    s6:begin
    seq_detected='b1;
    next_state=din?s2:s0;
    end
    default:next_state = s0;
endcase
end

always@(posedge clk or posedge rst)begin
  if(rst)begin
    current_state<=s0;
  end
else begin
  current_state<=next_state;
end
end

endmodule
```

Result and lesson. Both detectors pass streams containing back-to-back and overlapping occurrences; the Moore output appears one cycle after the final bit, exactly as Chapter 6 predicts. The lesson is design-before-code: drawing the fallback arcs correctly (each mismatch returns to the longest matching prefix) is the entire difficulty, and the three-block FSM idiom turned the diagram into RTL without further decisions.

8.9 Laboratory 10 — Stream parity generator using functions

Specification. Compute the even parity of a sliding 8-bit window over a serial stream, using a Verilog **function** for the parity reduction.



Figure 20: Laboratory 10 block diagram: each clock shifts `serial_in` into an 8-bit window; the parity function (a one-line XOR reduction) evaluates combinationally on the next window value and is registered onto `parity_out`.

stream-parity-generator/src/stream_parity_gen.v Zeyad1Khaled/NTI_Labs

```
`timescale 1ns/1ps
module stream_parity_gen (

  input clk,reset,
  input serial_in,
  output reg parity_out

);
reg [7:0] shift_register;

function parity (
  input [7:0]in
);
begin
parity=^in;
end
endfunction

always@(posedge clk)begin
  if(reset)begin
    shift_register <= 8'b0;
    parity_out<='b0;
  end
else begin
  shift_register<={shift_register[6:0],serial_in};
  parity_out <= parity({shift_register[6:0], serial_in});

end
end
endmodule
```

Result and lesson. The testbench streams random bits and compares against a testbench-side reference model. The lesson is the place of functions in synthesizable code: a zero-delay, single-value abstraction that keeps RTL readable without changing the hardware — here, a XOR tree over eight window bits.

8.10 Laboratory 11 — Switch debouncer and edge detectors

The session of 19–20 July added two small but formative designs: rising-edge detectors as Moore and Mealy machines (the Mealy version reacting in the same cycle, the Moore one cycle later — the table of Chapter 6 observed live), and a switch debouncer that filters a bouncing input with a saturation counter before registering the clean level. Both sit in the team repository's dated session folders and reinforced the same point: metastability-adjacent, real-world inputs demand deliberate synchronisation, not hope.

9 Capstone project: FPGA-based automotive ECU suite

9.1 Motivation and proposal

The final project of the course was submitted by a team of five — Amr Waheed, Aly Hamdy, Zeyad Khaled, Abdelrahman Mohamed and Adham Amr — under the NTI project-recognition scheme (proposal 19 July, submission 23 July). The team chose an **advanced automotive electronic control unit (ECU) suite**: safety- and performance-critical vehicle functions implemented as dedicated synchronous hardware instead of tasks on a shared microcontroller.

The engineering argument is the one the whole training builds toward. A sequential MCU executes `task 1 → task 2 → …` plus interrupts and OS jitter; acceptable for comfort functions, risky for safety functions, because queuing delay turns into variable reaction time. A late ABS reaction lengthens stopping distance; a missed rev limit damages an engine; a crashed task hangs everything. The suite's answer: one dedicated hardware controller per function, all executing in the same clock cycle — true parallelism, deterministic latency, and fault isolation.

9.2 Architecture

Figure Figure 21 shows the suite as built: five modules sharing the vehicle-level signals (speed, brake, throttle, RPM, echo) and producing actuator-level outputs. Design rules were fixed in the proposal: single clock, synchronous resets, fully synthesizable RTL, and a self-checking testbench per module.

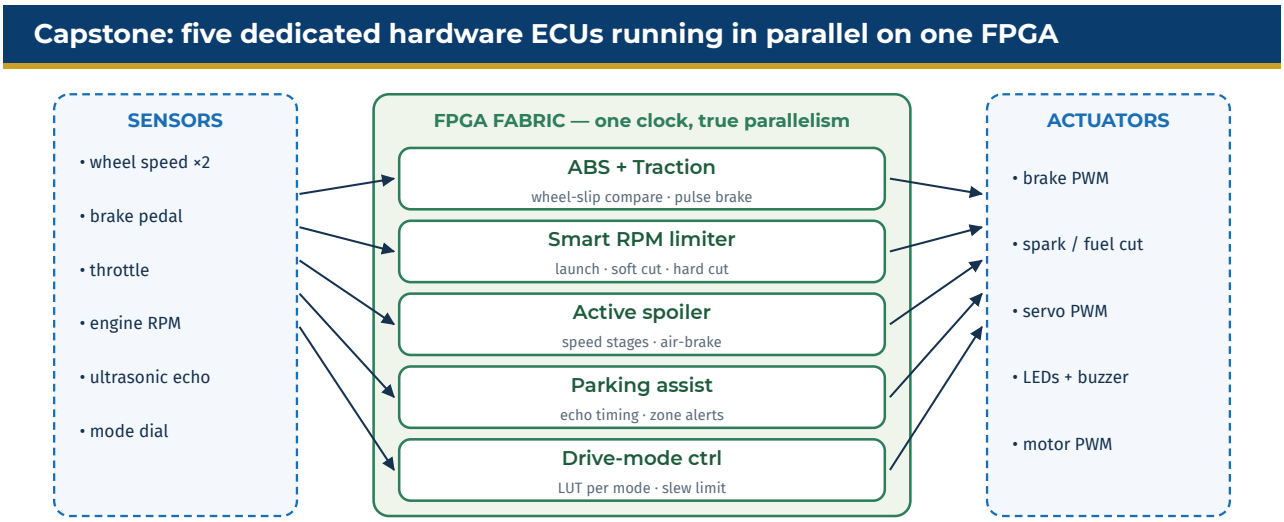


Figure 21: Top-level architecture of the capstone suite. Sensors on the left feed five independent synthesizable controllers inside one clocked FPGA fabric; actuators on the right receive PWM, cut and alert signals. No controller waits for another: the five blocks evaluate every cycle, concurrently.

Module	Inputs	Outputs	Testbench
ABS + traction control	front/rear wheel speed (8-bit), brake, ecu PWM	final PWM, ABS brake pulse, TCS/ABS LEDs	`tb_Traction_ABS`
Smart RPM limiter	engine RPM, speed, brake, throttle	spark retard, fuel cut	`tb_Smart_RPM`
Active spoiler	speed (9-bit), brake	servo PWM, air-brake LED	`tb_Active_Spoiler`
Ultrasonic parking assist	echo pulse	green/yellow/red LEDs, buzzer	`tb_parking_system`
Multi-mode drive controller	throttle (2-bit), mode (2-bit)	motor PWM (8-bit)	`tb_Drive_Mode`

Table 14: The five ECU modules, their interfaces and verification

9.3 Subsystem design notes

ABS + traction control. Every cycle the module compares front and rear wheel speed. Under braking with slip above the threshold (difference > 15) it generates an ABS pulse so the wheel never locks — modulating pressure in the increase→hold→decrease cycle; under drive with wheel spin it halves the torque PWM (traction control). The FSM of Chapter 6 appears here verbatim as the pulse sequencer.

Smart RPM limiter. A three-stage protection scheme: launch control near 4 000 rpm (vehicle stopped, brake and full throttle) holds revs by spark retard; a soft cut at 9 000 rpm retards spark in the warning zone; a hard cut at 10 000 rpm cuts fuel in the danger zone. Spark retard acts on the power stroke and fuel cut on the intake, so the redline is a hardware property, not a software promise.

Active spoiler. Speed stages map to servo targets — closed below 80 km/h, cruise angle 80–150 km/h, maximum downforce above — and braking at speed deploys the full air-brake (PWM 255) with its LED. A ±1-step slew limiter moves the servo smoothly, the same rate-limiting idiom used in the drive controller.

Ultrasonic parking assist. An always-on echo timer: a counter starts with the trigger, stops with the echo return, and the count maps to distance zones (safe, mid, close, danger) that escalate LED colour and buzzer rate from slow beeps to continuous.

Multi-mode drive controller. A lookup table per drive mode (Eco, Normal, Sport, Track) maps throttle steps to target duty cycles — from Eco's calm 1:2 mapping to Track's aggressive 3:1 — and a slew-rate limiter (±1 step per window) protects the drivetrain from step commands. Its duty output can also be handed to the traction module for torque management, which is how the suite's modules integrate without coupling.

`project/rtl/Drive_Mode_Controller.v (excerpt)` alyhamdy879/FPGA-labs-project

```
module drive_Mode_Controller (
    input wire clk,
    input wire rst,
    input wire [1:0] throttle_in,
    input wire [1:0] drive_mode,
    output reg motor_pwm
);

    reg [7:0] target_duty;
    reg [7:0] current_duty;

    reg [7:0] pwm_counter;
    reg [15:0] smooth_timer;

    // drive Modes
    always @(*) begin
        case (drive_mode)
            2'b01: begin // Normal 1:1 Ratio
                case (throttle_in)
                    2'b00: target_duty = 8'd0;
                    2'b01: target_duty = 8'd64;
                    2'b10: target_duty = 8'd128;
                    2'b11: target_duty = 8'd255;
                endcase
            end

            2'b00: begin // Eco 1:2 Ratio
                case (throttle_in)
                    2'b00: target_duty = 8'd0;
                    2'b01: target_duty = 8'd32;
                    2'b10: target_duty = 8'd64;
                    2'b11: target_duty = 8'd128;
                endcase
            end

            2'b10: begin // Sport
                case (throttle_in)
                    2'b00: target_duty = 8'd0;
                    2'b01: target_duty = 8'd128; // 2:1 Ratio
                    2'b10: target_duty = 8'd192; // 4:3 Ratio
                    2'b11: target_duty = 8'd255; // 1:1 Ratio
                endcase
            end

            2'b11: begin // Track
                case (throttle_in)
                    2'b00: target_duty = 8'd0;
                    2'b01: target_duty = 8'd192; // 3:1 Ratio
                    2'b10: target_duty = 8'd255; // 2:1 Ratio
                    2'b11: target_duty = 8'd255; // 1:1 Ratio
                endcase
            end

            default: target_duty = 8'd0;
        endcase
    end

    // Smooth Rate Limiter (Engine Protection)
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            current_duty <= 8'd0;
            smooth_timer <= 0;
        end
        else
        begin
             if (smooth_timer >= 50) begin // 50 * 255 * 10 ns = 127.5 us resonable for sim (irl shuold
be >50000 )
                smooth_timer <= 0;

                if (current_duty < target_duty)
                    current_duty <= current_duty + 1;
                else if (current_duty > target_duty)
                    current_duty <= current_duty - 1;
            end
            else begin
                smooth_timer <= smooth_timer + 1;
            end
        end
    end

    //PWM Signal
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            pwm_counter <= 8'd0;
```

```
            motor_pwm <= 1'b0;
        end
        else begin
            pwm_counter <= pwm_counter + 1;

            if (pwm_counter < current_duty)
                motor_pwm <= 1'b1;
            else
                motor_pwm <= 1'b0;
        end
    end

endmodule
```

9.4 Verification and results

Each module ships a self-checking testbench; all five report passing: lock-up braking, launch spin and cruising for ABS/TCS; a 0→10 000 rpm sweep plus the launch combination for the limiter; a speed ramp through all spoiler stages plus a brake stab at speed; short and long echo pulses for the parking assist; and every mode × throttle combination for the drive controller. Against the MCU baseline the suite offers parallel execution (five controllers in one cycle instead of five queued tasks), deterministic reaction time (identical every run), and isolation (one faulty block cannot hang the others). The hardware demo used a physical mode dial to switch Eco→Track live, and the design was reviewed and recognised under the NTI project scheme.

9.5 What the project consolidated

The capstone is where the training's threads tie together: specification-first design (Chapter 2), FSM controllers and datapaths (Chapter 6, Laboratories 6–9), synchronous coding rules (Chapter 5), PWM and counters as the actuator interface (Laboratory 8), self-checking verification (Chapter 5), and Git discipline across five contributors — with the timing vocabulary of Chapter 7 as the language of the design reviews.

10 Competencies acquired

The table maps each competency claimed by this report to the artefact that evidences it.

Competency	Level reached	Evidence
CMOS/IC fundamentals	Can explain device→gate→chip trade-offs (power, delay, area)	Chapter 4 course quick-checks
Verilog RTL	Fluent: styles, parameters, hierarchy, assignment semantics	All lab sources, Chapter 8
Sequential design	Counters, registers, RAMs, reset strategies without latches	Labs 5–8
FSM design	Diagram → three-block RTL, Moore/Mealy choice, encoding choice	Lab 9, capstone controllers
Verification	Self-checking testbenches, randomised stimulus, waveform reading	All testbenches; Figures Figure 13–Figure 14
Static timing analysis	Arcs, unateness, setup/hold, slack, XDC constraints	Chapter 7 XDC lab
Tooling	ModelSim/Questa projects, Icarus CLI, Vivado synthesis, VCD review	Chapter 3 toolchain table
Engineering practice	Spec-first work, Git versioning, code review, team delivery	Repositories; capstone of Chapter 9

Table 15: Competency map: what was learned and where it is evidenced

11 Evaluation of the experience

11.1 What worked well

The training's strongest feature is its **specification culture**. Every lab arrived as a written spec with a block diagram, port names and a verification expectation — the format industry uses — so the habit formed early of designing from a document rather than from a solution. The second strength is sequencing: each lecture block was immediately consumed by a lab that used it, which is why concepts like non-blocking assignment arrived as debugging lessons rather than as slides. The third is the verification emphasis: self-checking testbenches turned "it looks right on the waveform" into "it prints TEST PASSED", and the repository keeps that evidence reviewable. Finally, the team capstone reproduced a real engineering constraint — a fixed proposal date, a fixed submission date, five people, one repository — and the review session treated the designs as products, not homework.

11.2 Difficulties encountered, objectively

- [**Assignment semantics.**] The first datapath bug was a blocking assignment inside a clocked block: simulation and intent agreed, the synthesised structure did not. Fixing it cost an evening and produced the personal rule now stated in Section 5.3.
- [**Bidirectional buses.**] Laboratory 5's `inout` port fought every instinct: driving it from a procedural block compiled, simulated wrong, and only the net/variable model of Section 5.2 explained why.
- [**Handshake debugging.**] In Laboratory 4 a one-cycle-too-early `valid` corrupted roughly one word in a thousand; finding it required reading the waveform at the protocol level, not the code level — a skill the training explicitly set out to teach.
- [**Online delivery.**] The fully online mode saved travel and made recordings revisitable, but it taxed hands-on moments: no bench, no board for most of the programme, and debugging help arrived asynchronously. Mitigations adopted: office-hour slots, a shared team chat for waveform screenshots, and local FPGA demos at the end.
- [**Scope pressure.**] Five capstone modules in four days after a full lab week stretched the team; two modules were deliberately simplified (spoiler slew window, parking zone count) and the simplifications are documented in the project notes rather than hidden.

11.3 Reflection on my own performance

I attended all sessions, delivered every laboratory from its specification without reference solutions, and kept the repository commit-by-commit in session order — which is why Appendix C can be cross-checked against commit dates. My strongest contributions were the integrated datapath and the control-unit testbench; my weakest moment was underestimating the capstone's integration cost and discovering it on the proposal day. If I repeated the training I would start the project repository on day one and route every lab through the same continuous-integration habit I built only in week three. On balance the experience moved me from writing Verilog to **designing hardware in Verilog**: the language became a means, and the waveform became the argument.

12 Conclusion and future work

The 120-hour NTI × ITIDA training delivered what its title promises: a complete first pass over digital IC design as practised today. It began with the physics and economics of CMOS, crossed the abstraction layers through Verilog RTL, established verification and timing closure as non-negotiable disciplines, and ended with a team capstone that put five synthesizable controllers on an FPGA and proved them cycle-accurate. The laboratories, waveforms and code preserved in the accompanying repositories (Appendix Appendix A) are the durable product of that month; this report is its account.

The natural continuation is the back-end half of Figure Figure 4: physical design and sign-off in a real technology node, constraint-driven synthesis at scale, and formal verification; on the verification side, moving from self-checking testbenches to constrained-random methodologies (UVM); and on the hardware side, porting the capstone suite from simulation to a physical board with real sensors and actuators. The training ended, in the best sense, by making those next steps concrete and scheduled.

13 References

1. O. M. Aladawy, *Introduction to Digital IC Design*, lecture slides, NTI × ITIDA summer training, July 2026 (79 slides).
2. M. Salah, *Digital Design using FPGA — RTL Design Course*, lecture slides, NTI × ITIDA summer training, July 2026 (203 slides).
3. M. Salah, *Digital Design using FPGA* — Verilog depth sessions: data types, hierarchy, procedural blocks, blocking vs non-blocking (11–12 July); `case` variants, functions and tasks (13–14 July); finite-state machines (16 July), NTI, 2026.
4. M. Salah and A. Abdelazeem, *Static Timing Analysis* module: timing arcs and unateness, transmission gates, latches and flip-flops, setup/hold, timing paths and slack, NTI, July 2026.
5. NTI, *Semi-Project Laboratory Sheets 3–9 and the Counter Lab*: modelling an address multiplexor; modelling a data driver; modelling the ALU; modelling a controller; modelling a generic register; modelling a single-bidirectional-port memory; modelling a generic counter; designing a 2-bit up/down counter, NTI, July 2026.
6. NTI, *Final Project — Group Formation and Proposal Requirements*, announcement, 18 July 2026.
7. Z. K. Abd Elhamid, *NTI Labs* — laboratory repository: designs, testbenches and ModelSim projects for all ten labs, GitHub: `Zeyad1Khaled/NTI_Labs`, 2026.
8. A. Hamdy et al., *FPGA-labs-project* — session workspace and capstone suite (RTL, testbenches, XDC lab, project report), GitHub: `alyhamdy879/FPGA-labs-project`, 2026.
9. A. Waheed, A. Hamdy, Z. Khaled, A. Mohamed, A. Amr, *Advanced Automotive ECU: Performance Tuning and Safety Systems*, project proposal and report, NTI project recognition, July 2026.
10. IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2001 (Verilog-2001), and IEEE Std 1800 (SystemVerilog) — language references for the constructs used.
11. Mentor Graphics, *ModelSim/QuestaSim User Documentation*, v10.1d — simulation environment used throughout.

A Repository guide and reproduction

All laboratory artefacts are preserved in two public repositories:

Each project in `NTI_Labs` follows one layout — `src/` for RTL, `tb/` for testbenches, `sim/` for the prepared ModelSim/Questa project (`.mpf`) and waveform dumps — and can be simulated either by opening the `.mpf` in ModelSim/Questa or from the command line with Icarus Verilog:

Reproducing any laboratory simulation NTI_Labs README

```
# cd <project>
iverilog -o sim.vvp src/*.v tb/*.v
vvp sim.vvp   // self-checking benches print TEST PASSED
```

B Laboratory-to-repository mapping

C Daily training log

Hours per session as recorded in the training log; the total equals the contracted 120 hours.

Repository	Contents	Use
`Zeyad1Khaled/NTI_Labs`	Ten project directories (`src/`, `tb/`, `sim/` per project), README with layout and history, original submission archives	Primary evidence for Chapter 8
`alyhamdy879/FPGA-labs-project`	Dated session folders (11_7 … 22_7), lab PDFs, XDC lab, capstone `project/` with RTL, testbenches, presentation and report	Session log cross-check; Chapter 9

Table 16: Repositories accompanying this report

Session	Laboratory / deliverable	Repository path	Concepts
11 Jul	Lab 3 — address multiplexor	`NTI_Labs/multiplexer-2-to-1`	parameters, level-sensitive blocks
11 Jul	Lab 4 — tri-state driver	`NTI_Labs/tri-state-driver`	`z` state, replication
11 Jul	Lab 5 — ALU	`NTI_Labs/arithmetic-logic-unit`	operators, `case`
12 Jul	Lab 6 — VeriRISC controller	`NTI_Labs/microcontroller-controller`	phase decode, control table
12 Jul	Lab 7 — generic register	`NTI_Labs/microcontroller-register`	non-blocking, sync reset
13 Jul	`casex`/`casez` experiments	`FPGA-labs-project/13_7`	wildcard matching
13–14 Jul	Lab 10 — stream parity (functions)	`NTI_Labs/stream-parity-generator`	functions in RTL
14–15 Jul	Lab 8 — bidirectional memory	`NTI_Labs/bidirectional-memory`	`inout`, arbitration
14–15 Jul	Lab 9 — generic counter	`FPGA-labs-project/14_7`	blocking + non-blocking mix
16 Jul	FSM lab — Moore/Mealy machines	`FPGA-labs-project/16_7`	state diagrams, encoding
19 Jul	Counter in three styles; edge detectors; 110101 detector	`NTI_Labs/up-down-counter`, `NTI_Labs/sequence-detector-110101`, `FPGA-labs-project/19_7`	modelling styles, FSM
20 Jul	Switch debouncer	`FPGA-labs-project/20_7`	synchronisation, saturation counter
22 Jul	STA constraint lab (XDC)	`FPGA-labs-project/22_7`	clocks, delays, slack
19–23 Jul	Capstone ECU suite (team)	`FPGA-labs-project/project`	full flow

Table 17: Full mapping of sessions, laboratories, sources and concepts

Date	Session content	Hours
Mon 6 Jul	Opening; course overview; IC history, Moore's law, ASIC vs FPGA	4
Tue 7 Jul	Semiconductor and CMOS fundamentals; inverter, VTC, noise margins	4
Tue 8 Jul	Power dissipation; transmission gates, latches, flip-flops; timing parameters	4
Thu 9 Jul	ASIC flow; FPGA architecture and flow; clocks and timing intro	4
Fri 10 Jul	RTL course: gates, number systems, module syntax, modelling styles	4
Sat 11 Jul	Data types, hierarchy, procedural blocks, BA vs NBA; Labs 3–5 (MUX, driver, ALU)	5
Sun 12 Jul	Labs 6–7 (controller, register); controller testbench completion	5
Mon 13 Jul	BA/NBA review; `casex`/`casez`; functions; parity lab start	5
Tue 14 Jul	Tasks (static vs automatic, disable); counter and memory labs	5
Wed 15 Jul	Memory testbench review; parity generator with generate; lab close-out	4
Thu 16 Jul	FSM lecture: Moore/Mealy, diagrams, ASM, encoding; FSM lab	5
Sun 19 Jul	Counters in three styles; Moore/Mealy edge detectors; sequence detector; **project proposal**	5
Mon 20 Jul	Sequence detector completion; debouncer; testbench hardening	4
Tue 21 Jul	Capstone implementation: ABS/TCS and RPM limiter modules	4
Wed 22 Jul	STA lecture: arcs, unateness, setup/hold, paths, slack; XDC lab	5
Thu 23 Jul	Capstone implementation: spoiler, parking, drive-mode modules; **submission**	5
Sun 26 Jul	Capstone verification: testbench completion and waveform review	4
Mon 27 Jul	Capstone verification: randomised stimulus and fixes	4
Tue 28 Jul	Project presentation and demo video preparation	4
Wed 29 Jul	Repository restructure: layouts, READMEs, portable ModelSim paths	4
Thu 30 Jul	Repository review with supervisor; corrections applied	4
Sat 1 Aug	Report writing: scientific chapters and figure drafting	4
Sun 2 Aug	Report writing: laboratory chapter with waveforms	4
Mon 3 Aug	Report writing: project chapter, evaluation, appendices	4
Tue 4 Aug	Full re-simulation of all labs for report evidence	4
Wed 5 Aug	Report review against faculty instructions; formatting pass	4
Thu 6 Aug	Discussion preparation; final review; **training close-out**	3
Total		**120**

Table 18: Day-by-day log of the training (6 July – 6 August 2026)